

ANTHROPIC • CLAUDE CERTIFICATION PROGRAM

CLAUDE CERTIFIED ARCHITECT

Foundations (CCAR-F) — The Complete Deep-Dive Study Guide for Agentic Architecture, Tool & MCP Design, Claude Code, Prompt Engineering, and Context Reliability.

5 DOMAINS • FULL COVERAGE

PASS SCORE 720/1000

2026 EDITION

WHAT THIS GUIDE COVERS

Domain 1 — 27%

Agentic Architecture & Orchestration

Domain 3 — 20%

Claude Code Configuration & Workflows

Domain 5 — 15%

Context Management & Reliability

Domain 2 — 18%

Tool Design & MCP Integration

Domain 4 — 20%

Prompt Engineering & Structured Output

Applied

The Six Official Exam Scenario Contexts

AUTHORED BY

Mohan Seerangan

Complete Certification Study Guide
Chapter-wise · Syllabus-aligned · Deep-dive edition

Table of Contents

Complete chapter-wise map of this guide, aligned to the official CCAR-F exam blueprint. Page numbers refer to this PDF.

—	How to Use This Guide & Exam Blueprint	4
—	Sources & Confidence	6
01	Foundations — Claude, the Agentic Loop & the Architect Mindset	8
	1.1–1.4 Core mechanics: messages API, stop_reason, product surface	8
	1.5 The prompts-vs-hooks governing rule	12
	1.6 Worked example: assembling the loop end-to-end	13
	1.7 Selecting a model for a given agent role	14
02	Domain 1 — Agentic Architecture & Orchestration (27%)	15
	2.1–2.2 The agentic loop & task decomposition	15
	2.3–2.4 Orchestrator–subagent model & topologies	17
	2.5–2.7 Hooks, idempotency & loop termination	19
	2.8 Deep dive: diagrams & code examples	21
	2.9 Domain 1 exam-ready summary	23
03	Domain 2 — Tool Design & MCP Integration (18%)	24
	3.1–3.3 Tool anatomy, tool_choice & isError semantics	24
	3.4–3.6 MCP architecture, primitives & least privilege	27
	3.7–3.8 Deep dive & trust boundaries / tool-result safety	29
	3.9 Domain 2 exam-ready summary	30
04	Domain 3 — Claude Code Configuration & Workflows (20%)	31
	4.1–4.3 CLAUDE.md, config hierarchy & .mcp.json vs ~/.claude.json	31
	4.4–4.6 Permissions, subagents & headless CI/CD mode	34
	4.7 Claude Agent SDK for custom agents	36
	4.8–4.9 Deep dive walkthrough & exam-ready summary	37
05	Domain 4 — Prompt Engineering & Structured Output (20%)	38
	5.1–5.3 System prompts, few-shot examples & chain-of-thought	38
	5.4–5.5 Structured output & error-handling limits	41
	5.6 Prompt caching & the Batch API	42
	5.7–5.8 Deep dive & exam-ready summary	43
06	Domain 5 — Context Management & Reliability (15%)	45
	6.1–6.2 Context window & compaction / case-facts pattern	45
	6.3–6.4 Lost-in-the-middle, conflicts & escalation triggers	46

6.5–6.7 Grounding, deep dive & retrieval strategies	47
6.8 Domain 5 exam-ready summary	49
07 Applied Architecture — The Six Official Scenario Contexts	50
7.1 Customer Support Resolution Agent	50
7.2 Code Generation with Claude Code	51
7.3 Multi-Agent Research System	52
7.4 Developer Productivity with Claude	53
7.5 Claude Code for CI/CD	54
7.6 Structured Data Extraction	55
08 Exam Readiness — Final Review System	56
8.1 The distractor-recognition cribsheet (all 5 domains)	56
8.2 Master decision-frameworks index	58
8.3 Complete glossary of exam terminology	60
8.4 Out-of-scope topics & exam-day logistics	63
8.5 Final 24-hour review checklist	64
09 Reference Appendix — Complete Technical Reference	65
9.1–9.2 Messages API & stop_reason reference	65
9.3–9.6 Claude Code, hooks, MCP & topology reference	67
9.7–9.8 Cost/latency reference & cross-domain synthesis	70
10 Pattern Catalog — Fifteen Named Architecture Patterns	72
11 Anti-Pattern Catalog — Recognized Failure Patterns	75
12 Quick-Reference Cards — One Card Per Domain	78
13 Configuration Templates — Production-Ready References	84

How to Use This Guide & Exam Blueprint

Read this first — it sets up everything that follows

This guide is a complete, self-contained study resource for the **Claude Certified Architect — Foundations (CCAR-F)** exam, Anthropic's official credential validating the ability to design, implement, and reason about production-grade systems built with Claude. It is organized **chapter-by-chapter around the five official exam domains**, in the exact proportions Anthropic uses to weight the exam, so that your study time maps directly onto your score potential.

EXAM SNAPSHOT

Certification	Claude Certified Architect — Foundations (CCAR-F)
Format	60 scenario-based, single-answer multiple-choice questions, drawn from 4 of 6 official scenario contexts (randomized per sitting)
Duration	120 minutes, proctored, cannot be paused once started
Scoring	Scaled 100–1000; passing score is 720/1000 (not a raw percentage)
Penalty for wrong answers	None — there is no penalty for guessing, so never leave an item blank
Validity	Credential is valid for 12 months from the date of passing
Prerequisites	None formally required; practical experience with the Claude API, Claude Code, and MCP is strongly recommended

Exam Domain Weighting

Five domains make up 100% of the scored content. The single biggest lever for your score is respecting these weights in your study plan — Domain 1 alone is worth more than Domains 2 and 5 combined.

#	Domain	Weight	Guide Chapter
1	Agentic Architecture & Orchestration	27%	Chapter 2
2	Tool Design & MCP Integration	18%	Chapter 3
3	Claude Code Configuration & Workflows	20%	Chapter 4
4	Prompt Engineering & Structured Output	20%	Chapter 5
5	Context Management & Reliability	15%	Chapter 6

HOW TO STUDY THIS GUIDE

1. **Read Chapter 1 first.** It gives you the mental model — the *agentic loop* and the *prompts-are-probabilistic, hooks-are-deterministic* frame — that every later chapter builds on. Skipping it makes every other domain harder to retain.
2. **Study domains in weight order** ($1 \rightarrow 3/4 \rightarrow 2 \rightarrow 5$). If your time is limited, this order protects the most points per hour invested.
3. **Use the boxed callouts.** Amber boxes are exam tips, blue boxes are conceptual grounding, green boxes are hard rules the exam treats as absolute, red boxes are common failure patterns, and purple boxes flag the exact distractor logic the exam uses.
4. **Finish with Chapter 7** to see every domain applied inside the six official scenario contexts, then use Chapter 8 as your final compression pass in the 24 hours before your exam.

WHAT THIS EXAM DOES NOT TEST

The exam is scenario-based architectural judgment, not API trivia or model-internals theory. It deliberately excludes: Constitutional AI/RLHF theory, fine-tuning, embedding models, the Computer Use and Vision APIs, low-level streaming/SSE mechanics, OAuth/API-key rotation internals, rate-limit and quota arithmetic, token-pricing memorization, and cloud-provider-specific configuration. This guide does not spend pages on those topics for exactly that reason — every page here maps to something scored.

Sources & Confidence

What is confirmed, what is cross-validated, and what remains unverifiable — stated plainly

This page exists so you can calibrate your trust in this guide the same way a careful architect calibrates trust in any source: by knowing exactly what was confirmed, how, and what still carries some uncertainty. Nothing below is hidden in the chapters that follow — it is summarized here for one clear reference point.

What Is Directly Confirmed

CONFIRMED FROM ANTHROPIC'S OWN PUBLIC ANNOUNCEMENT

- Claude Certified Architect, Foundations (CCAR-F) is a **real, currently-live technical certification**, launched by Anthropic as part of the Claude Partner Network — not a hypothetical or discontinued program.
- It is explicitly described by Anthropic as **"a technical exam for solution architects building production applications with Claude."**

What Is Cross-Validated Across Independent Sources

CONFIRMED BY AGREEMENT BETWEEN TWO INDEPENDENT THIRD-PARTY SOURCES

The domain structure, exam format, and scoring model used throughout this guide were checked against a second, independent source (a third-party exam-preparation tool built around this certification). That source independently reported:

- A 60-question exam scored on a 0–1000 scale with a **passing score of 720** — matching this guide exactly.
- A five-domain structure with a question distribution of **D1:16, D2:11, D3:12, D4:12, D5:9** — which converts to the same **27% / 18% / 20% / 20% / 15%** weighting used to size Chapters 2–6 of this guide.
- The same five domain names and general scope: Agentic Architecture & Orchestration, Tool Design & MCP Integration, Claude Code Configuration & Workflows, Prompt Engineering & Structured Output, and Context Management & Reliability.

Two independently-arrived-at sources reporting identical figures is meaningfully stronger evidence than either source alone — this is the basis for treating the domain weighting in this guide as reliable.

What Remains Unverified

STATED HONESTLY, NOT GLOSSED OVER

- Anthropic does not appear to have published a fully public, official candidate handbook or syllabus document for CCAR-F — the certification is delivered through the partner-gated Claude Partner Network, not as an open public exam.
- The specific domain names, task-statement wording, and question distribution in this guide are therefore based on **the best publicly available and cross-validated secondary evidence**, not a document directly authored and published by Anthropic for candidates.
- Exam programs can change scope, weighting, or format over time. If you have access to an official candidate handbook through a partner-program login, treat that document as authoritative over this guide wherever the two differ, and use this guide primarily for its conceptual and architectural depth.

What This Means for How You Should Use This Guide

PRACTICAL GUIDANCE

- **Trust the architectural content fully.** The technical explanations of agentic loops, tool design, MCP, Claude Code configuration, prompt engineering, and context management describe how these systems actually work, independent of exam-syllabus sourcing questions.
- **Trust the domain weighting with high, cross-validated confidence** — it is corroborated by two independent sources rather than asserted from one.
- **Treat exam logistics (format, scoring scale, passing threshold) as well-supported** but confirm against any official partner-portal material you can access before your exam date, since these details are the most likely to change over time.

Claude, the Agentic Loop & the Architect Mindset

The mental model every domain depends on

1.1 What Claude Certified Architect Actually Tests

Every CCAR-F question is a **scenario**: a description of a system someone is building with Claude, a design decision that needs to be made, and four answer options that all sound plausible. The exam is not asking "what does this parameter do" — it is asking "given this production constraint, which architectural choice correctly trades off reliability, cost, latency, and safety." Three habits of mind separate a passing candidate from a failing one:

- **Read for the constraint, not the topic.** A question about tool design might actually be testing context management if the real issue is a tool returning too much data. Identify what breaks the system, then match the domain concept that fixes it.
- **Prefer deterministic controls over probabilistic ones whenever a single failure is costly.** This is the single highest-yield heuristic on the exam and is explained fully in section 1.5.
- **Never invent information the scenario doesn't give you.** Reliable architectures — and reliable exam answers — preserve uncertainty rather than guessing. If two options both seem defensible, the correct one is almost always the one that does *less* unjustified assuming.

1.2 The Messages API and the Core Request/Response Cycle

Every Claude-based system, no matter how sophisticated the orchestration layer around it, is built on the same primitive: a call to the **Messages API**. Understanding this cycle precisely is foundational because agentic loops, subagents, hooks, and MCP servers are all just structured ways of repeating and intercepting this cycle.

A request contains, at minimum: a `model`, a `messages` array alternating `user` and `assistant` turns, an optional `system` prompt (top-level, not part of the messages array), a `max_tokens` budget, and — when the agent needs to act on the world — a `tools` array describing the functions Claude may call.

```
{
  "model": "claude-sonnet-5",
  "max_tokens": 1024,
  "system": "You are a support triage agent...",
  "messages": [
    { "role": "user", "content": "My order #4821 never arrived." }
  ],
  "tools": [ { "name": "lookup_order", "description": "...", "input_schema": {...} } ]
}
```

The response is not free text alone — it is a **content array** that can mix **text** blocks and **tool_use** blocks. An architect must design the calling code to iterate this array and branch on block type, never assume the response is a single string. This is the number one implementation bug that CCAR-F scenario questions probe.

1.3 stop_reason — The Single Most Important Field on the Exam

Every response carries a **stop_reason** field, and correctly branching on it is the backbone of every agentic loop. Anthropic's own domain breakdown calls this out explicitly as the highest-frequency concept tested across all five domains — it appears as a distractor even in questions nominally about tools, context, or Claude Code.

stop_reason	Meaning	Architect's correct action
end_turn	Claude finished its turn naturally, no tool call pending	Return the response to the user; loop ends
tool_use	Claude wants to call one or more tools before continuing	Execute the tool(s), append a tool_result message, and send another request — the loop continues
max_tokens	The response was truncated by the token budget	Treat as incomplete; do not parse partial JSON as final — raise the budget or truncate the task, then retry
stop_sequence	A custom stop sequence was matched	Expected/intentional cutoff — parse up to the sequence
pause_turn	Claude paused a long-running turn (e.g., extended tool use) and can be resumed	Re-send the paused response back to continue rather than treating it as finished
refusal	Claude declined to continue for safety reasons	Surface this distinctly — do not retry the same request expecting a different result; escalate or reroute

DISTRACTOR PATTERN TO RECOGNIZE

The exam frequently offers an answer that treats `tool_use` as a terminal state ("the agent is done, return the text to the user") — this is always wrong. `tool_use` is a mid-loop signal, not an end state. Any option that skips executing the tool and returning a `tool_result` is a distractor.

The Core Agentic Loop, Precisely

Combining 1.2 and 1.3 gives the loop every agentic system in this guide is built from:

1. **Send** the current message history (plus system prompt and tool definitions) to the Messages API.
2. **Check** `stop_reason`.
3. If `tool_use`: **execute** the requested tool(s) in your own application code, **append** the assistant's `tool_use` block and a corresponding `tool_result` block to the message history, and **go to step 1**.
4. If `end_turn` (or another terminal reason): return the result and stop.

Chapter 2 builds the entire domain of agentic architecture on top of this loop — decomposition, subagents, hooks, and idempotency are all mechanisms for making this loop safe, observable, and boundable at production scale.

1.4 Claude's Product & Model Surface

The exam draws scenarios from across Anthropic's product surface, and confusing these layers is a common source of wrong answers. An architect needs to know which layer a given capability lives in.

Layer	What it is	When an architect reaches for it
Claude API (Messages API)	The raw HTTP interface to Claude models; the foundation everything else is built on	Any custom application, backend service, or integration
Claude Agent SDK	A framework (formerly "Claude Code SDK") for building custom autonomous agents with built-in tool execution, permissions, and the agentic loop already wired up	Building a bespoke agent product beyond a simple chat wrapper
Claude Code	Anthropic's own agentic coding tool — a CLI/IDE-integrated product built on the Agent SDK, pre-configured for software engineering workflows	Developer productivity, codebase work, CI/CD automation
Model Context Protocol (MCP)	An open protocol standardizing how agents discover and call external tools/resources across any host application	Connecting Claude (in any of the above layers) to external systems in a reusable, host-agnostic way

MENTAL SHORTCUT

API = the engine. Agent SDK = the chassis you build a custom vehicle on. Claude Code = a finished vehicle Anthropic already built (for coding). MCP = the universal fuel-and-parts standard all of them can plug into. A scenario that says "we built our own support agent from scratch" is Agent SDK / raw API territory; one that says "our developers use it inside their terminal to edit a repo" is Claude Code territory.

1.5 The Prompts-vs-Hooks Frame That Unlocks Every Domain

If you remember exactly one idea from this entire guide, it should be this one, because it resolves a huge share of CCAR-F's trickiest scenario questions across all five domains:

THE GOVERNING RULE

Prompts are probabilistic. Hooks (and other host-side, code-enforced controls) are deterministic. A system prompt instruction like "never delete files without confirmation" is a strong influence on behavior, but it is still a probability, not a guarantee — Claude is a language model, and any instruction delivered purely in natural language can, in principle, be missed, mis-weighted against a conflicting instruction, or fall outside the training distribution for an edge case. A **hook**, permission rule, or other piece of surrounding application code that *intercepts* an action before or after it happens is enforced by ordinary software, not by model behavior, so it cannot be talked around.

The architectural consequence: **whenever a single failure would cause financial loss, a security breach, irreversible data damage, or a compliance violation, the correct control is a deterministic one** — a hook, a permission boundary, a schema validator, a human approval gate in code — never a prompt instruction alone, no matter how carefully worded. Prompts remain the right tool for shaping quality, tone, and reasoning approach, where being right "almost always" is an acceptable bar.

Risk level	Example	Correct control type
Low — quality/ tone only	Keep responses under 200 words; use a formal tone	Prompt instruction is sufficient
Medium — recoverable error	Prefer summarizing before quoting a source	Prompt instruction, optionally reinforced with a validation step
High — irreversible/ costly	Never run <code>rm -rf</code> ; never issue a refund over \$500 without approval; never call a production-deploy tool from a test branch	Hook or permission rule — deterministic, code-level enforcement

HOW THIS SHOWS UP AS A DISTRACTOR

A question describes a costly failure mode and offers an answer that "strengthens the system prompt" or "adds a more explicit instruction." This is almost always the wrong choice when a hook-based or permission-based alternative is also offered — the exam is testing whether you reach for enforcement, not persuasion, once the stakes cross into irreversible territory.

1.6 Worked Example: Assembling the Loop End-to-End

To make sections 1.2–1.5 concrete before Chapter 2 builds on them, walk through a minimal but complete example: an agent that checks an order status and, if the order is delayed,

drafts an apology email. This example is intentionally small — the goal is to see every concept from this chapter cooperate in one place.



On the first call, Claude receives the user's message ("is my order #4821 delayed?") along with a `lookup_order` tool definition. Because the agent cannot answer without checking the order system, Claude responds with `stop_reason: "tool_use"` and a `tool_use` content block naming `lookup_order` with `{"order_id": "4821"}` as arguments. Per section 1.3, this is a mid-loop signal, not a final answer — the calling application must not show this to the user as-is.

```
// Application code executes the requested tool
result = lookup_order(order_id="4821")
// result: { status: "delayed", eta: "2026-09-02" }

// Application appends the tool_use block AND a new tool_result block,
// then sends the whole updated history back to the Messages API
messages.append({"role": "assistant", "content": [tool_use_block]})
messages.append({"role": "user", "content": [
  {"type": "tool_result", "tool_use_id": tool_use_block.id,
   "content": '{"status":"delayed","eta":"2026-09-02"}', "is_error": false}
]})
```

The second call returns `stop_reason: "end_turn"` with a plain-text apology draft, because the agent now has everything it needs and no further action is required. Notice `is_error: false` — the lookup succeeded even though the news (a delay) is not what the customer wanted to hear; a successful call with an unwelcome-but-valid result is never an error (this exact distinction returns as its own topic in section 3.3).

Now overlay section 1.5: if this same agent also had a `send_email` tool, would sending the apology automatically be safe to leave purely to the system prompt's judgment? Per the governing rule, sending an external customer communication is a real-world, semi-irreversible action (the customer receives it). A production version of this agent would pair the prompt instruction ("draft an empathetic apology") with a **PreToolUse hook on `send_email`** that, at minimum, logs every send for audit — and, if the business risk warrants it, requires a human approval step before the call executes. Chapter 2 formalizes exactly this reasoning as a general architecture, not a one-off email example.

1.7 Selecting a Model for a Given Agent Role

Not every agent or subagent in a system needs the same model, and matching model capability to task complexity is a real architectural decision the exam expects you to reason about, even though it avoids testing specific pricing figures.

Task profile	Reasoning
High-volume, low-complexity classification or routing subagent (e.g., "is this ticket billing or technical?")	A smaller, faster model is usually sufficient and meaningfully cheaper at scale — the task doesn't require deep multi-step reasoning
General-purpose orchestrator or customer-facing agent	A mid-tier, broadly capable model balances quality and cost for typical production traffic
Complex multi-constraint reasoning, architectural planning, or highest-stakes decisions	The most capable available model is justified when the cost of a wrong decision clearly outweighs the marginal cost difference between models

EXAM FRAMING

A scenario describing a hub-and-spoke research system might use a strong model for the orchestrator's synthesis step (where reasoning quality matters most) and a lighter model for simple, repetitive subagent lookups — mixed-model architectures are a legitimate, exam-recognized pattern, not a shortcut to be avoided.

WHAT TO CARRY FORWARD

Every concept in Domain 1 through Domain 5 is, at its core, a variation on this five-step loop: what gets decomposed before step 1, who is allowed to see what across steps 3–4, what deterministic control wraps step 3 for risky actions, how the message history in step 4 is kept from growing unbounded over many iterations, and what condition eventually stops step 5 from repeating forever. Keep this worked example in mind as a reference point as you move through the next five chapters.

Agentic Architecture & Orchestration

The largest domain on the exam — master this chapter first

2.1 The Agentic Loop: Gather Context → Take Action → Verify → Repeat

Chapter 1 established the mechanical loop (send → check stop_reason → execute tool → repeat). Domain 1 asks you to reason about the **architectural loop** layered on top of it — the pattern every reliable agent follows regardless of implementation: **gather context, take action, verify the result, and repeat until the goal is met or a boundary is hit.**

Phase	What happens	Common architectural failure
Gather context	The agent retrieves the information it needs — via tool calls, retrieved documents, prior conversation state — before acting	Acting on stale or incomplete context because a retrieval step was skipped to save latency
Take action	The agent calls a tool, writes a file, sends a message, or otherwise changes the state of the world	Taking an irreversible action without a verification or approval gate proportional to its risk
Verify	The agent (or surrounding code) checks that the action produced the intended effect before considering the sub-goal complete	Trusting a tool's "success" response without checking the actual resulting state — the tool call succeeding is not the same as the goal being achieved
Repeat / terminate	The loop continues toward the next sub-goal, or stops because the task is complete or a boundary condition is reached	No explicit termination condition, causing indefinite looping (see 2.7)

EXAM FRAMING

Scenario questions about "the agent kept retrying the same failing action" or "the agent reported success but nothing actually changed" are testing the **verify** phase. The fix is almost never "improve the prompt" — it is adding an explicit verification step in the surrounding code that checks real-world state, independent of what the model claims.

Why This Loop Is the Backbone of the Whole Domain

Every other Domain 1 concept is a refinement of one phase of this loop at scale: decomposition determines how a big goal is broken into loop iterations; the orchestrator-subagent model determines who runs which loop iteration and with what context; hooks determine what deterministically happens around the "take action" phase; idempotency determines what happens when "repeat" accidentally re-executes something; and termination controls determine when "repeat" must stop.

2.2 Task Decomposition and When NOT to Decompose

Decomposition is the practice of breaking a complex goal into smaller sub-tasks that can be solved (and often delegated) independently. It is powerful, but the exam tests the judgment of knowing when decomposition helps versus when it adds coordination overhead without benefit.

When decomposition helps

- The sub-tasks are largely **independent** and can be verified separately (e.g., "research competitor A," "research competitor B," "research competitor C").
- Different sub-tasks benefit from **different tool access or context** — isolating them keeps each agent's context window focused and its blast radius limited.
- The overall task is long enough that a single agent's context window would otherwise fill with irrelevant intermediate detail.

When decomposition hurts

- The sub-tasks are **tightly coupled** and require constant shared context — splitting them forces expensive round-trips of information between agents that a single agent would have had natively in one context window.
- The task is **simple enough** that one well-scoped call to the agentic loop solves it; adding a coordinator layer only adds latency and failure surface.
- Decomposition is used to paper over a **missing tool or missing context** rather than an actual coordination need — this is one of the most tested anti-patterns.

FAILURE-TRACING RULE (HEAVILY TESTED)

When an agent-based system produces incomplete or missing coverage of a task (e.g., a research report that skips an entire topic), the root cause is almost always **decomposition at the coordinator level** — the coordinator failed to identify that a topic needed a sub-task — *not* a failure inside any individual subagent. Diagnose top-down: check what the orchestrator planned before inspecting what any subagent executed.

2.3 Orchestrator–Subagent Model & Context Isolation

The orchestrator–subagent pattern assigns one agent (the **orchestrator** or **coordinator**) the job of planning and delegating, while one or more **subagents** each execute a bounded piece of work in their own isolated context window.

Role	Responsibility	Context scope
Orchestrator	Interprets the overall goal, decomposes it into sub-tasks, dispatches each to a subagent, collects and synthesizes results, decides when the goal is complete	Sees the high-level plan and each subagent's <i>summarized result</i> — not each subagent's full internal working context
Subagent	Executes one bounded sub-task using only the tools and context relevant to that sub-task	Sees only what it needs for its sub-task; its internal tool calls, intermediate reasoning, and raw tool outputs stay isolated from the orchestrator and from sibling subagents

Why context isolation is the point, not a side effect

The single biggest reason to use subagents is **context isolation**: it keeps the orchestrator's context window free of noisy intermediate detail (raw search results, failed tool attempts, verbose tool schemas) that a subagent generated while doing its job. Only the subagent's *final, synthesized result* crosses back into the orchestrator's context. This has two consequences the exam probes directly:

- **Errors inside a subagent don't automatically pollute the orchestrator's reasoning** — a subagent can retry, backtrack, or explore a dead end internally without the orchestrator ever seeing that noise, as long as the subagent's final report is clean.
- **The orchestrator cannot see what it wasn't told.** If a subagent's summary omits an important caveat or partial failure, the orchestrator has no way to know — which is why subagent result contracts (what a subagent is required to report, including partial failures) matter architecturally, not just as a nicety.

DISTRACTOR PATTERN

A question describing "the orchestrator made a bad decision because it didn't know a subagent had failed halfway through" is testing whether you recognize a **missing structured result contract** from the subagent — the fix is requiring subagents to explicitly report partial failure/success status, not giving the orchestrator raw access to the subagent's internal transcript (which would defeat the purpose of isolation).

2.4 Multi-Agent Topologies

Beyond the basic orchestrator/subagent pair, production systems arrange multiple agents into one of several recognized topologies. Knowing which topology fits which problem shape is directly tested.

Topology	Shape	Best fit	Key risk
Hub-and-spoke	One central orchestrator dispatches to and collects from many independent subagents; subagents never talk to each other directly	Parallelizable, largely independent sub-tasks (e.g., multi-topic research, batch document processing)	Orchestrator becomes a bottleneck/single point of failure if it also does heavy synthesis work
Pipeline	Agents are chained in sequence; each agent's output becomes the next agent's input	Tasks with a natural sequential stage structure (e.g., extract → validate → transform → load)	A single stage's failure blocks the entire pipeline unless explicit error-handling/retry stages exist between links
Peer-to-peer	Agents communicate directly with each other, without a central coordinator	Rare on the exam; appropriate only for tightly-coupled negotiation-style tasks where a hub would be an unnecessary hop	Hardest to observe, debug, and bound — coordination logic is implicit rather than centralized, and is generally the <i>least</i> preferred topology for production reliability

DEFAULT ANSWER HEURISTIC

When a CCAR-F question doesn't strongly signal sequential dependency, **hub-and-spoke is the safer default** — it keeps coordination logic centralized, observable, and easiest to add deterministic controls (hooks, budgets, approval gates) to at a single point. Prefer it unless the scenario clearly describes staged, sequential processing (favoring pipeline) or genuine peer negotiation (favoring peer-to-peer, rarely correct).

Agent Communication & Handoff

Regardless of topology, agent-to-agent communication should be a **structured handoff**, not a raw conversation dump: a defined schema (goal, relevant facts, constraints, expected output format) rather than the full prior transcript. This keeps the receiving agent's context focused and prevents compounding token growth across a multi-agent chain — directly reinforcing the context-isolation principle from 2.3.

2.5 Hooks: PreToolUse, PostToolUse & the Deterministic Control Layer

Hooks are code that the host application runs automatically at defined points in the agentic loop, outside of and independent from the model's own reasoning. They are the concrete mechanism behind the "deterministic control" side of the prompts-vs-hooks rule from Chapter 1.

Hook point	Fires	Typical architectural use
PreToolUse	Before a requested tool call is actually executed	Block or modify a disallowed call (e.g., reject a file-delete outside an approved directory); enforce parameter validation; require human approval for high-risk calls
PostToolUse	After a tool call has executed and returned a result	Log the action for audit; run an automated verification check on the result; trigger a notification; sanitize output before it re-enters the model's context

Because hooks run in ordinary application code, they can inspect and reject a call *before* it happens — something no prompt instruction can guarantee, since the model always retains some (small) probability of attempting the disallowed action anyway. This is why hooks are the correct answer whenever a scenario says a single mistaken action would be unacceptable.

HOW TO RECOGNIZE A "SHOULD BE A HOOK" SCENARIO

- The action is **irreversible** (deletion, financial transaction, external message send, production deploy).
- The failure mode is described as a **compliance, security, or safety** requirement rather than a quality preference.
- The scenario mentions wanting an **audit trail** or guaranteed logging — logging via hook is deterministic; logging "by asking Claude to log it" is not.

2.6 Idempotency, Retries & Failure Tracing

Agentic loops retry. Retries against a non-idempotent action (one that has a different effect if run twice) are a classic production hazard: a tool call that "charges a customer" or "sends an email" must not silently double-execute just because the agent, or the surrounding retry logic, re-issued the same tool call after an ambiguous or timed-out response.

IDEMPOTENCY KEY PATTERN

Design tools that perform state-changing, side-effecting actions to accept an **idempotency key** — a unique identifier generated once per logical operation and passed with every retry of that same operation. The receiving system checks the key and, if it has already processed that key, returns the prior result instead of re-executing the effect. This is the standard, exam-correct pattern for making retries safe without banning retries outright (which would sacrifice reliability for a problem that has a better fix).

Failure tracing is the discipline of determining *where* in a multi-step or multi-agent system a fault actually originated, rather than where it was *observed*. As established in 2.2, an incomplete or wrong final result is often a decomposition/coordination failure at the orchestrator level even though it surfaces as "the final report was wrong" — always trace upstream from the symptom to the planning or hand-off step that actually produced the gap before concluding a subagent's internal logic was at fault.

2.7 Loop Termination, Budgets & Runaway-Agent Controls

An agentic loop needs an explicit reason to stop, beyond "the model eventually returns end_turn," because a model can also loop indefinitely between tool calls (attempting a failing action repeatedly, or expanding scope beyond the original goal). Production architectures always layer host-enforced boundaries on top of the model's own judgment:

- **Maximum iteration / turn count** — hard-cap the number of loop iterations (tool-call round-trips) per task, independent of what the model "wants" to do next.
- **Token or cost budget** — cap cumulative tokens or spend per task and terminate (with a clear partial-result report) once exceeded.
- **Wall-clock timeout** — bound total elapsed time, particularly important for any user-facing or CI-integrated agent.
- **Repeated-failure circuit breaker** — if the same tool call fails N times in a row (a strong signal of a stuck loop), stop and escalate rather than retrying indefinitely.

DISTRACTOR TO AVOID

"Add a prompt instruction telling Claude to stop after a reasonable number of attempts" is a probabilistic control applied to a problem (runaway resource consumption) that demands a deterministic one. The correct answer is always a host-enforced counter/budget/timeout, per the Chapter 1 governing rule.

2.8 Deep Dive: Visualizing the Orchestrator–Subagent Model

The following hub-and-spoke diagram represents the multi-agent research pattern referenced throughout this domain and revisited in Chapter 7. Study the direction of the arrows carefully: information flows *down* as a scoped sub-task assignment, and flows *up* only as a synthesized, structured result — never as a raw transcript.



Subagents A, B, and C never communicate with one another directly in a hub-and-spoke topology — this is precisely what makes the topology easy to reason about and easy to observe: every cross-agent interaction passes through the orchestrator, where deterministic controls (budgets, hooks, approval gates) can be applied at a single, centralized point rather than needing to be replicated at every edge of a denser communication graph.

Deep Dive: A Minimal Hook Configuration

Concretely, a hook is registered declaratively (in Claude Code, inside `settings.json`; in a custom Agent SDK application, via the SDK's hook registration API) and bound to a specific event and, optionally, a specific tool pattern:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "issue_refund",
        "command": "scripts/require_approval_if_over_50.sh"
      }
    ],
    "PostToolUse": [
      {
        "matcher": "*",
        "command": "scripts/audit_log.sh"
      }
    ]
  }
}
```

Two details of this example are exam-relevant beyond the syntax itself. First, the `matcher` field lets a hook target a specific tool (`issue_refund`) or every tool call (`*`) — precise matchers are how you apply the “hooks for costly actions only” principle without over-constraining low-risk tools. Second, the `PreToolUse` hook here can *reject* the call outright (returning a non-zero exit / blocking response) before `issue_refund` ever executes — this is

the exact mechanism, not just the concept, behind "a hook can guarantee what a prompt can only request."

Deep Dive: A Structured Subagent Result Contract

Section 2.3 argued that subagents must report status explicitly rather than the orchestrator inferring it. A minimal result contract looks like this:

```
{  
  "subagent_id": "research-competitor-b",  
  "status": "partial_success",  
  "summary": "Found pricing and feature data; could not confirm 2026 headcount.",  
  "unresolved": ["2026 headcount figure unavailable from public sources"]  
}
```

The `status` and `unresolved` fields are what make this contract useful: without them, a subagent that silently degraded from a complete answer to a partial one looks, from the orchestrator's point of view, identical to one that fully succeeded — and the orchestrator would synthesize a final report that confidently omits a caveat it was never told existed. This is the concrete implementation of the "don't invent information" principle applied to inter-agent communication specifically.

2.9 Domain 1 Exam-Ready Summary

Concept	One-line takeaway
Agentic loop	Gather context → act → verify → repeat; "verify" checks real state, not the model's claim of success
stop_reason	<code>tool_use</code> is mid-loop, never terminal; always execute the tool and continue
Decomposition	Split only genuinely independent/decoupled sub-tasks; missing coverage traces to coordinator planning, not subagents
Orchestrator/ subagent	Isolation keeps noisy internal work out of the orchestrator; require structured result contracts, including partial failure
Topologies	Hub-and-spoke default; pipeline for sequential stages; peer-to-peer rarely correct
Hooks	PreToolUse blocks/validates before execution; PostToolUse verifies/logs after — use whenever a single failure is costly
Idempotency	Idempotency keys make retries of state-changing actions safe without disabling retries
Termination	Iteration caps, budgets, timeouts, and failure circuit breakers must be host-enforced, not prompt-requested

Tool Design & MCP Integration

How Claude discovers, chooses, and safely uses external capabilities

3.1 Anatomy of a Tool: Descriptions Are the Routing Layer

A tool definition has three parts: a `name`, a `description`, and an `input_schema` (JSON Schema describing the parameters). The exam's central insight about tool design is this:

THE DESCRIPTION IS NOT DOCUMENTATION — IT IS THE ROUTING MECHANISM

Claude decides *whether* to call a tool, *which* tool among several candidates to call, and *how* to fill its parameters, based almost entirely on the natural-language description at inference time. A minimal or vague description ("gets data") does not just look unprofessional — it measurably degrades tool selection accuracy, because the model has nothing to disambiguate this tool from similar ones. Treat writing tool descriptions as a prompt-engineering task, not a documentation afterthought.

A well-formed description states: **what the tool does, when to use it** (and, if relevant, when not to), **what each parameter means** including format/units, and **what the tool returns**. Parameter names and descriptions inside `input_schema` matter just as much as the top-level description — a parameter named `id` with no description invites malformed calls; `order_id (the 6-digit numeric order identifier shown on the customer's receipt)` does not.

DISTRACTOR PATTERN

A scenario where "the agent kept calling the wrong tool" or "the agent used the right tool with malformed parameters" is almost always a **tool description quality** problem, not a model-capability problem. The tempting-but-wrong fix is switching models or adding a prompt instruction listing every tool by name; the correct fix is rewriting the tool's own description and parameter documentation.

3.2 tool_choice: auto, any, tool, none

The `tool_choice` parameter controls how strongly Claude is directed to use tools on a given turn, and picking the wrong mode is a frequently-tested error.

Value	Behavior	Use when
<code>auto</code>	Claude decides for itself whether a tool call, plain text, or both is appropriate	Default for general-purpose agents where not every turn requires tool use
<code>any</code>	Claude must call <i>some</i> tool, but may choose which one	The turn's purpose is guaranteed to require action (e.g., a step in a pipeline that always performs a lookup), but which specific tool depends on input
<code>tool</code> (naming a specific tool)	Claude is forced to call exactly the named tool	You need a guaranteed, schema-shaped output and are using tool-calling purely as a structured-output mechanism (see 5.4)
<code>none</code>	Claude is prevented from calling any tool this turn, even if tools are defined	You want Claude's plain-text reasoning or a final summary turn without triggering another action

COMMON MISTAKE

Setting `tool_choice: any` when the correct mode is `auto` forces a tool call even on turns where the right behavior is a plain-text clarifying question — this produces forced, low-quality tool calls with fabricated parameters just to satisfy the constraint. Reserve `any` /named-`tool` for turns where action is structurally guaranteed.

3.3 Tool Results, isError Semantics & Empty-Result Handling

Every tool result is returned to Claude as a `tool_result` block, which can be flagged with `is_error`. Getting this flag right is a small mechanical detail with an outsized effect on agent reliability, and the exam tests it precisely:

Situation	Correct <code>is_error</code> value	Why
The tool executed successfully and legitimately found zero matching results (e.g., a search that found no orders for that ID)	<code>is_error: false</code>	Zero results is a valid, successful outcome, not a failure — Claude needs to reason about "nothing was found" differently than "the tool broke"
The tool call failed to execute (bad parameters, downstream service down, permission denied, timeout)	<code>is_error: true</code>	Claude needs an explicit signal that the attempt itself failed, so it can retry, use a different tool, or surface the failure rather than treating an error message as data

WHY THIS IS HEAVILY TESTED

Mislabeling an empty-but-successful result as an error (or vice versa) causes Claude to draw the wrong conclusion — treating "no orders found" as a system failure worth retrying (wasting loop iterations) or treating a genuine failure as valid data (fabricating a response from an error string). This single boolean is one of the highest-leverage, easiest-to-get-wrong details in tool design.

3.4 Model Context Protocol: Architecture, Hosts, Clients, Servers

The Model Context Protocol (MCP) is an open, standardized protocol for connecting AI applications to external tools, data, and prompts — designed so that a tool integration written once can be reused across any MCP-compatible host, instead of being reimplemented per application.

Role	Definition	Example
Host	The user-facing application that embeds an MCP client and orchestrates the overall interaction with Claude	Claude Code, Claude Desktop, a custom internal agent application
Client	The component (usually one per connected server) inside the host that speaks the MCP protocol and manages a 1:1 connection to a server	The MCP client module inside Claude Code
Server	A separate process (local or remote) that exposes tools, resources, and/or prompts over MCP	A GitHub MCP server, a company's internal ticketing-system MCP server

WHY THIS ARCHITECTURE MATTERS TO AN ARCHITECT

Because the host-client-server boundary is a real process boundary, MCP gives you a natural place to apply least-privilege access control (3.6): a host can connect to exactly the servers it needs, and each server can be scoped to expose only the tools appropriate for that integration — without any of that scoping logic living inside the model's prompt.

3.5 MCP Primitives: Tools, Resources, Prompts & Transports

Primitive	Purpose	Analogy
Tools	Model-invoked functions the server exposes, callable by Claude mid-loop, same concept as native API tool use but delivered over MCP	A function call
Resources	Application-controlled, addressable data the host can read and inject into context (e.g., a file, a database record) — typically surfaced by the host UI/logic rather than invoked autonomously by the model the way tools are	A readable file or URI
Prompts	Reusable, server-defined prompt templates that a host can surface to users (e.g., as slash commands) to kick off a common workflow with pre-filled structure	A saved, parameterized prompt template

Transports

MCP servers communicate with clients over one of two main transport mechanisms: **stdio** (the server runs as a local subprocess, communicating over standard input/output — simplest, ideal for local tools) and **HTTP-based transport** (the server runs as a remote, possibly shared service, reachable over the network — necessary when the server is centrally hosted or used by multiple clients/hosts). Choosing between them is a scenario-based judgment: local developer tools and anything touching the local filesystem favor stdio; shared enterprise integrations reachable by many users/hosts favor a remote HTTP-based server.

3.6 Scoped Tool Access & Least-Privilege Agent Design

A recurring anti-pattern the exam tests is giving a single agent (or subagent) unrestricted access to every tool available in the environment — for example, connecting a customer-support agent to eighteen tools when its task only ever needs four or five. This is architecturally wrong for reasons beyond tidiness:

- **Larger tool lists degrade selection accuracy.** More candidate tools, especially similar ones, increases the chance Claude picks a plausible-but-wrong tool — directly compounding the description-quality issue from 3.1.
- **Blast radius.** An agent that can only call `lookup_order` and `issue_refund_under_50` cannot be tricked (by a prompt injection, a bug, or a rare model error) into calling a tool with a bigger blast radius, because that tool was never exposed to it in the first place. Scoping is a security control, not just a UX one.

- **Matches the orchestrator/subagent model.** Each subagent in a multi-agent system (Chapter 2) should receive only the tools relevant to its bounded sub-task — a research subagent gets search tools, a writing subagent gets none.

RULE OF THUMB

Scope each agent/subagent to roughly **4–5 tools directly relevant to its task**, not the full tool catalog available in the system. If a scenario mentions an agent with a long, undifferentiated tool list and describes selection errors or safety concerns, the fix is reducing the exposed tool surface for that agent — this is almost always the correct answer over "add clearer instructions about which tools to use."

3.7 Deep Dive: MCP Message Flow & Tool Schema Anatomy

Section 3.4 introduced the host/client/server roles. Here is the same architecture as a request flow, which is the shape most CCAR-F scenario questions actually describe even when they never use the word "MCP" explicitly:



When Claude decides to call a tool exposed by an MCP server, the host's client translates that into an MCP protocol message sent to the server, the server executes the actual integration logic against the external system, and the result is translated back into a standard `tool_result` that re-enters Claude's context exactly as if it were a natively-defined tool. This is precisely why MCP-exposed tools are subject to the exact same description-quality (3.1), `tool_choice` (3.2), and `is_error` (3.3) principles as any other tool — from the model's perspective, there is no architectural difference between a native tool and an MCP-exposed one.

Deep Dive: Anatomy of a Well-Formed Tool Schema

```
{
  "name": "lookup_order",
  "description": "Retrieves the current status, ETA, and line items for a single customer order. Use this whenever the customer references an order number or asks about delivery status. Returns a not-found result (not an error) if the order ID does not exist.",
  "input_schema": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string",
        "description": "The 4-6 digit numeric order identifier shown on the customer's receipt or confirmation email."
      }
    },
    "required": ["order_id"]
  }
}
```

Three details here directly reflect section 3.1's rule that descriptions are a routing mechanism, not documentation: the top-level description states *when* to use the tool ("whenever the customer references an order number"), pre-empts a common confusion by clarifying that a not-found result is not an error (reinforcing 3.3 at the schema level itself), and the parameter description gives a concrete format ("4-6 digit numeric") rather than leaving Claude to guess what a well-formed `order_id` looks like.

3.8 Trust Boundaries & Tool-Result Content Safety

A subtler Domain 2 topic concerns what happens when a tool's *output* — not just its call — carries risk. A tool result (whether from an MCP server or a native tool) is text that re-enters Claude's context exactly like any other content, which means a malicious or compromised data source returning content designed to look like new instructions (a prompt-injection attempt embedded in a document, webpage, or ticket body) is architecturally the same problem as any other untrusted input.

- **Treat tool results as data, not instructions.** Application-level design should make clear (structurally, e.g., via role/formatting) that tool_result content is retrieved information to reason about, not a new set of commands to follow — the same distinction from Chapter 1 between the system prompt (trusted) and everything else (context to evaluate, not blindly obey).
- **Least privilege compounds as a security control here too.** An agent scoped to read-only tools (3.6) cannot be manipulated into taking a damaging write action via a poisoned tool result, because the write-capable tool was never exposed to it in the first place — this is

the same reasoning as 3.6, now applied to an adversarial-input threat model rather than just an accuracy one.

- **Deterministic gates still apply to model-influenced decisions.** If a tool result could plausibly influence Claude toward a costly action (e.g., a scraped webpage suggesting "you should now transfer funds to..."), the same PreToolUse hook principle from 2.5 protects against a manipulated model decision exactly as it protects against a mistaken one — the hook doesn't care why the model requested a disallowed call, only that it was requested.

3.9 Domain 2 Exam-Ready Summary

Concept	One-line takeaway
Tool descriptions	The description and parameter docs <i>are</i> the routing mechanism — write them as carefully as a prompt
tool_choice	<code>auto</code> default; <code>any</code> /named <code>tool</code> only when action is structurally guaranteed each turn
isError	<code>false</code> for a successful call with zero results; <code>true</code> only for an actual execution failure
MCP roles	Host embeds a client; client connects 1:1 to a server; server exposes tools/resources/prompts
MCP primitives	Tools = model-invoked functions; Resources = host-surfaced readable data; Prompts = reusable templates
Transports	stdio for local subprocess tools; HTTP-based for shared/remote servers
Least privilege	Scope each agent to ~4–5 relevant tools; never expose the full catalog to every agent

Claude Code Configuration & Workflows

Environment setup, permissions, subagents, and CI/CD integration

4.1 Claude Code Architecture & the CLAUDE.md Memory System

Claude Code is Anthropic's agentic coding tool, built on the Claude Agent SDK, that runs the agentic loop (Chapter 1–2) against a real codebase with real file-system and shell tools. Its most important architectural feature for the exam is **CLAUDE.md** — a plain-Markdown file that Claude Code automatically reads into context at the start of a session to give the agent durable project memory without repeating instructions every prompt.

CLAUDE.md typically documents	Why it belongs here rather than in a one-off prompt
Build/test/lint commands	Needed on nearly every session; repeating them per-prompt wastes tokens and invites drift
Code style and architectural conventions	Should be consistently applied across the whole team's sessions, not re-explained by each developer
Repository structure and key module boundaries	Orients a fresh agent session quickly, reducing wasted exploration turns
Known gotchas / things not to do	Encodes team lessons-learned once, persistently

LAYERED CLAUDE.MD FILES

CLAUDE.md can exist at multiple levels — a repository root file for project-wide conventions, and nested CLAUDE.md files in subdirectories for module-specific context that is only loaded when Claude Code is actually working in that area. This mirrors the configuration-hierarchy principle in 4.2: broad defaults at the top, specific overrides closer to where they apply.

4.2 Configuration Hierarchy: User, Project, and Enterprise Scope

Claude Code settings resolve through a layered hierarchy, and knowing which layer wins — and which layer is appropriate for a given kind of setting — is directly tested.

Scope	Typical location	Shared via git?	Use for
Enterprise / managed	Centrally deployed policy settings	N/A — deployed by org admins	Org-wide mandatory security policy (e.g., disallowed tools, mandatory hooks) that individual users cannot override
Project	<code>.claude/</code> inside the repository	Yes — committed to version control	Settings the whole team should share: project-level MCP servers, project CLAUDE.md, shared slash commands
User / personal	<code>~/.claude/</code> in the user's home directory	No — never committed, personal to that machine/user	Individual preferences, personal API credentials, personal MCP servers not relevant to the whole team

THE PATH-PREFIX RULE (HIGHEST-YIELD FACT IN THIS DOMAIN)

You can resolve almost any Domain 3 configuration question from the path alone: **anything under `~/.claude/` is personal and never git-tracked; anything under a project's own `.claude/` directory is shared with the team via git.** A scenario describing "the same command behaves differently for different team members" is a signature symptom of a setting that was placed at user-level when it needed to be project-level (or vice-versa) — diagnose by asking which path prefix the setting lives under.

4.3 .mcp.json vs ~/.claude.json — The Path-Prefix Rule Applied to MCP

The same hierarchy principle governs how MCP servers are registered with Claude Code specifically:

File	Scope	Committed to git	Correct use case
<code>.mcp.json</code> (project root)	Project-level — every team member working in this repo gets these MCP servers automatically	Yes	A shared internal ticketing-system server, a shared documentation server the whole team should use for this codebase
<code>~/.claude.json</code> (home directory)	User/personal-level — applies across all of that user's projects, not tied to any one repo	No	A personal MCP server (e.g., the user's own calendar), or a server the user wants everywhere regardless of project

DISTRACTOR PATTERN

A scenario states an MCP server needs to be available to *every developer who clones the repository*, and offers an answer that configures it in `~/.claude.json`. This is always wrong — personal-scope files are never shared by cloning a repo. The correct answer configures it in the project's `.mcp.json` so it travels with the codebase.

4.4 Permissions, allowedTools & the settings.json Model

Claude Code's permission system is the concrete, product-level implementation of the least-privilege principle from Domain 2 and the deterministic-control principle from Domain 1. Settings such as `allowedTools` (and its complement, disallowed/denied tools) live in `settings.json` at the appropriate scope (project or user) and constrain what the agent is permitted to execute — enforced by the host application, not requested via prompt.

- **Allow-list model:** explicitly enumerate the tools/commands an agent may run in a given project (e.g., allow `git status`, `git diff`, deny arbitrary shell execution) — this is the deterministic, code-enforced equivalent of the "never delete files" prompt instruction from Chapter 1, and is the exam-correct choice whenever a scenario needs a guarantee rather than a strong suggestion.
- **Interactive approval:** for actions that are risky but still sometimes necessary, Claude Code can be configured to pause and request explicit human approval before executing, rather than either blanket-allowing or blanket-denying.
- **Project vs. user permission settings:** a security-critical permission boundary (e.g., "never allow force-push to main") belongs at project scope so it applies to every contributor and is enforced by policy, not personal configuration choice.

4.5 Slash Commands, Subagents & context: fork

Slash commands are reusable, named prompt templates (often backed by MCP prompts or local project files) that let a team standardize common workflows (e.g., `/review-pr`, `/write-tests`) so every developer invokes the same well-tuned instruction rather than freehand prompting each time.

Claude Code also supports spawning **subagents** for sub-tasks, applying the exact orchestrator/subagent pattern from Chapter 2 inside a coding session — for example, dispatching a subagent to run and summarize a noisy test suite so the main session's context isn't flooded with raw test output.

CONTEXT: FORK

A subagent (or a skill invoked within Claude Code) can be configured with `context: fork`, meaning it runs in an isolated context window that does not merge its full internal transcript back into the parent session — only its final summarized result does. This is the direct product-level mechanism for the context-isolation principle from section 2.3, and is the correct choice whenever a sub-task is expected to be noisy (many exploratory tool calls, long tool outputs) but only its conclusion matters to the parent session.

4.6 Headless Mode, the -p Flag & CI/CD Integration

Claude Code can run **headless** — without its interactive terminal UI — driven entirely by a single non-interactive invocation, which is the pattern used to embed it inside automated pipelines.

Element	Purpose
<code>-p</code> (print/non-interactive) flag	Runs Claude Code with a single prompt, prints the result, and exits — no interactive session, no waiting for further human input. This is the mechanism that makes Claude Code usable as a CI/CD pipeline step.
Exit codes / structured output	Headless invocations should be designed to return a machine-checkable result (exit code, or structured JSON output) so a CI pipeline can branch on success/failure automatically, rather than requiring a human to read prose output
Scoped permissions in CI	A CI-invoked Claude Code session should run with a tightly scoped allow-list (4.4) appropriate to an unattended, non-interactive context — CI is exactly the kind of "no human in the loop to approve" scenario where deterministic tool restriction matters most

EXAM FRAMING

A scenario describing "we want Claude Code to automatically review every pull request and post comments, with no human triggering it manually" is describing headless/ `-p` usage inside a CI/CD trigger (e.g., a pull-request webhook), combined with a scoped, non-interactive permission profile. Any answer that assumes an interactive terminal session or manual approval prompts is incompatible with the described automation.

4.7 The Claude Agent SDK for Custom Agent Applications

When a team needs an agent product that Claude Code does not already provide off the shelf (i.e., not a coding assistant, but some other autonomous application), the **Claude Agent SDK** is the correct building block, rather than reimplementing the agentic loop from raw API calls or trying to repurpose Claude Code itself for a non-coding domain.

- The SDK provides the agentic loop (Chapter 1), tool execution plumbing, permission/hook infrastructure, and subagent support already wired together — an architect uses it to avoid re-building this scaffolding from scratch for every new agent product.
- It is the same underlying framework Claude Code itself is built on, which is why concepts like hooks, permissions, and subagents transfer directly between "building a custom agent with the SDK" and "configuring Claude Code."
- Choosing SDK vs. raw Messages API is a build-effort trade-off: raw API calls give full control but require building the loop, tool-execution, and safety scaffolding yourself; the Agent SDK trades a little flexibility for a large amount of already-solved infrastructure.

Deep Dive: Agent SDK Scaffolding, Conceptually

While exact SDK syntax evolves, the conceptual shape an architect should recognize is stable and directly mirrors Chapter 1's loop plus Chapter 2's controls:

```
agent = ClaudeAgent(  
    system_prompt="You are a research assistant...",  
    tools=[search_tool, fetch_tool],           # scoped, per 3.6  
    permissions={"allow": ["search", "fetch"]}, # deterministic, per 1.5  
    hooks={"PreToolUse": [audit_hook]},        # per 2.5  
    max_iterations=25,                         # termination cap, per 2.7  
)  
result = agent.run("Summarize competitor pricing for Q3.")
```

Every constructor argument above corresponds to a decision this guide has already taught you to make deliberately: which tools (least privilege), which permissions (deterministic gating), which hooks (audit/verification), and what iteration ceiling (runaway-loop

protection). The SDK's value is that these are first-class, structured configuration — not something bolted on ad hoc around raw API calls.

4.8 Deep Dive: A Complete Project Configuration Walkthrough

To tie sections 4.1–4.7 together, walk through what a well-configured repository actually contains, file by file, and why each piece lives where it does.

```
my-project/
├─ CLAUDE.md           # project-wide memory, committed
├─ .mcp.json           # shared MCP servers, committed
├─ .claude/
│  └─ settings.json    # shared permissions/hooks, committed
│     └─ commands/
│        └─ review-pr.md # shared slash command, committed
└─ src/
   └─ billing/
      └─ CLAUDE.md     # module-specific memory, committed

~/.claude/
├─ settings.json       # personal overrides, NEVER committed
└─ ~/.claude.json      # personal MCP servers, NEVER committed
```

Notice that every file inside the repository's own `.claude/` directory (and the repo-root `CLAUDE.md` / `.mcp.json`) is committed and therefore identical for every developer who clones the repo — this is what guarantees consistent teammate behavior. Everything under the user's home-directory `~/.claude/` is invisible to git entirely, by design, because it is specific to that person's machine, not that project.

Deep Dive: A Representative settings.json

```
{
  "allowedTools": ["Bash(git diff:*)", "Bash(npm test:*)", "Read", "Edit"],
  "deniedTools": ["Bash(rm -rf:*)", "Bash(git push --force:*)"],
  "hooks": {
    "PreToolUse": [{ "matcher": "Bash(git push:*)", "command": "scripts/
confirm_branch.sh" }]
  }
}
```

Read this file the same way you would answer a Domain 3 scenario question: the allow-list scopes the agent to exactly the shell commands a coding task needs (least privilege, Domain 2's principle applied at the Claude Code product layer); the deny-list makes two catastrophic commands structurally unreachable regardless of what any prompt says (the Chapter 1 governing rule, implemented); and the hook adds a deterministic check on a risky-

but-sometimes-necessary command (a force-push-adjacent push) rather than either blanket-allowing or blanket-banning it.

Deep Dive: A Minimal CLAUDE.md

```
# Project: billing-service

## Commands
- Test: `npm test`
- Lint: `npm run lint`
- Build: `npm run build`

## Conventions
- All monetary values are integer cents, never floats.
- New endpoints require an OpenAPI schema update in /docs/api.

## Gotchas
- The `legacy_refunds` table is read-only; write path is now `refunds_v2`.
```

Every line in this file answers a question a fresh Claude Code session would otherwise have to discover the hard way (by trial, by reading source, or by asking) — that is the entire value proposition of CLAUDE.md: convert one-time team knowledge into zero-marginal-cost context for every future session.

4.9 Domain 3 Exam-Ready Summary

Concept	One-line takeaway
CLAUDE.md	Durable, auto-loaded project memory; can be layered by directory
Config hierarchy	Enterprise (mandatory) → Project <code>.claude/</code> (shared, git) → User <code>~/.claude/</code> (personal, never git)
MCP registration	<code>.mcp.json</code> = project-shared; <code>~/.claude.json</code> = personal, cross-project
Permissions	<code>allowedTools</code> /deny-lists in <code>settings.json</code> are deterministic, host-enforced controls
Subagents & context:fork	Isolate noisy sub-work; only the final summary returns to the parent session
Headless / -p	Non-interactive invocation is what makes Claude Code embeddable in CI/CD
Agent SDK	Use it to build custom (non-coding) agent products without re-implementing loop/permission/hook scaffolding

Prompt Engineering & Structured Output

Reliable instructions, reliable formats, and cost-aware execution

5.1 System Prompts, Role Framing & Explicit Categorical Criteria

The system prompt sets durable context: role, scope of responsibility, tone, and constraints that should hold across the entire session rather than being restated per-turn. The single highest-yield prompt-engineering principle tested on the exam concerns *how precisely* criteria are stated:

EXPLICIT CATEGORICAL CRITERIA BEAT VAGUE QUALIFIERS

An instruction like "be conservative when approving refunds" is a vague qualifier — it gives the model no fixed line to test each case against, so behavior drifts across similar-looking cases. An instruction like "approve automatically only if: (a) order is within 30 days, (b) amount is under \$50, and (c) the customer has no more than one prior refund this year; otherwise escalate" gives an explicit, checkable rule. The exam consistently rewards the option that replaces subjective language with an enumerable rule, because enumerable rules produce consistent behavior across many independent invocations — which is what a production system actually needs.

This principle also interacts with Chapter 1's prompts-vs-hooks rule: an explicit categorical rule in the prompt improves the model's reasoning consistency, but for the subset of that rule where a violation would be costly (e.g., never approve over \$50 automatically, ever), the boundary should *additionally* be enforced by a hook or permission check — the prompt rule and the hook are complementary, not substitutes for one another.

5.2 Few-Shot Examples: When They Help and When They Hurt

Few-shot examples (showing the model 2–5 worked examples of input→output pairs) are a powerful technique, but the exam specifically tests knowing where they add value versus where they are wasted effort or actively confusing.

Situation	Few-shot value	Why
Borderline/ambiguous cases that are hard to describe in a rule (e.g., tone calibration, edge-case categorization)	High — this is the ideal use case	Examples communicate a pattern that is difficult to state as an explicit rule; borderline cases are exactly where the model needs a concrete anchor
Obvious, unambiguous cases the model already handles correctly zero-shot	Low to negative	Adding examples for cases the model doesn't struggle with burns context budget without improving accuracy, and can even bias the model toward over-indexing on the literal example pattern instead of generalizing
Strict output formatting requirements	Medium-high , but structured output tooling (5.4) is usually superior	A well-chosen example clarifies format, but a schema-enforced tool call guarantees it — prefer the guarantee when available

DISTRACTOR PATTERN

A scenario shows a prompt already performing well on typical, obvious inputs, and offers "add several few-shot examples of typical cases" as a fix for a failure that only occurs on rare, edge-case inputs. This is a distractor — few-shot examples of the *already-working* case do not help the *edge* case; the correct answer targets the edge case specifically, either with an example of that edge case or an explicit rule covering it.

5.3 Chain-of-Thought & Extended Thinking for Architectural Reasoning

Prompting Claude to reason step-by-step before producing a final answer (chain-of-thought) improves performance on tasks requiring multi-step logic, and Claude's extended thinking capability formalizes this by giving the model a dedicated reasoning phase before its final response. Architecturally relevant judgment calls:

- **Use it for genuinely multi-step or judgment-heavy tasks** (complex eligibility decisions, multi-constraint scheduling, debugging) where skipping intermediate reasoning measurably increases error rate.
- **Skip it for simple, single-step lookups or formatting tasks** where it only adds latency and cost without a corresponding accuracy gain.

- **Keep reasoning and final output separated** in the response structure so downstream code can reliably parse the final answer without needing to strip out reasoning text — this matters directly for the structured-output patterns in 5.4.

5.4 Structured Output: Tool-Forced JSON vs. Prompt-Only JSON

Getting Claude to return data in a specific machine-readable shape (typically JSON) can be attempted two ways, and the exam tests which one provides an actual guarantee versus a strong tendency.

Approach	Mechanism	Guarantee level
Prompt-only JSON	Instructing Claude in natural language to "respond only in valid JSON matching this shape"	Probabilistic — usually correct, but can still emit stray prose, comments, trailing commas, or a subtly wrong shape; requires a parsing/validation step regardless
Tool-forced structured output	Defining the desired shape as a tool's <code>input_schema</code> and forcing that specific tool via <code>tool_choice</code> (3.2), so Claude's "response" is itself the schema-validated tool call arguments	Much stronger — the arguments are constrained to match the declared JSON Schema at generation time, eliminating most syntax-level errors

KEY NUANCE: SYNTAX ERRORS VS. SEMANTIC ERRORS

Tool-forced structured output reliably eliminates *syntax*-level errors (malformed JSON, missing required fields, wrong types) because the schema constrains what can be generated. It does **not** eliminate *semantic* errors — the model can still populate a syntactically perfect field with a factually wrong or poorly-reasoned value. Any exam option implying that switching to tool-based structured output "guarantees correctness" (rather than "guarantees shape") is a distractor.

5.5 Error-Handling Patterns: Retries, Validation Loops & Their Limits

Beyond forcing structure at generation time, production systems add a validation-and-retry loop: validate the model's output against the schema (and any business rules), and if it fails, send the failure back to Claude with a corrective instruction and ask it to try again.

THE LIMIT OF RETRY-BASED CORRECTION

A retry/validation loop can fix *malformed* output (wrong shape, wrong type, missing field) because the correction gives the model new, concrete information about what to fix. It **cannot reliably fix** a case where the model's answer is wrong because *the needed information was never in the source material Claude was given* — retrying does not manufacture facts that were never provided. The correct fix for that failure mode is upstream: better context/retrieval (Chapter 6), not another retry attempt at the generation step.

This distinction — "the loop can fix the model's mistake, not the pipeline's missing information" — is one of the most frequently tested judgment calls in Domain 4, because it requires correctly diagnosing *why* an output was wrong before choosing a fix.

5.6 Prompt Caching & the Batch API — Cost/Latency Trade-Offs

Prompt caching lets a long, unchanging portion of a prompt (e.g., a large system prompt, a lengthy set of tool definitions, or a big reference document) be cached on Anthropic's side so subsequent requests reusing that same prefix are cheaper and faster than reprocessing it from scratch every call. It is the correct optimization whenever an architecture repeatedly sends a large, stable prefix across many requests (e.g., an agent that calls the same tool-heavy system prompt hundreds of times per day).

The **Batch API** is a separate cost/latency trade-off, and the exam tests its constraints precisely, since candidates often assume it is a drop-in replacement for the standard API:

Property	Batch API	Standard (real-time) API
Cost	~50% discount versus standard pricing	Full price
Latency / SLA	No real-time latency guarantee — results are asynchronous and may take substantially longer (processed as capacity allows)	Synchronous, near-immediate response
Multi-turn tool calling	Not suited for interactive, multi-turn agentic tool-calling loops — each request is processed independently in a batch, not as part of a live back-and-forth	Native fit for the agentic loop from Chapter 1
Best fit	Large, non-urgent, independent workloads: bulk classification, bulk summarization, offline evaluation runs	Any user-facing or time-sensitive agentic workflow

DISTRACTOR PATTERN

A scenario describing a live, user-facing support agent that must respond within seconds, followed by an option suggesting "switch to the Batch API to cut costs," is a classic distractor — the cost saving is real, but the latency/no-multi-turn constraints make Batch fundamentally incompatible with a live, interactive agent. Batch is correct only when the scenario describes a large, non-urgent, non-interactive workload.

5.7 Deep Dive: Contrasting a Weak Prompt With a Strong One

The clearest way to internalize section 5.1's rule is to see the same requirement written twice.

WEAK — VAGUE QUALIFIER

"You are a support agent. Be conservative when approving refunds and use good judgment about escalating difficult cases."

STRONG — EXPLICIT CATEGORICAL CRITERIA

"Approve a refund automatically only if ALL of the following hold: (1) the order was placed within the last 30 days, (2) the requested amount is under \$50, and (3) the customer has received no more than one prior refund in the past 12 months. If any condition fails, or if the customer disputes the facts of the order, escalate to a human agent — do not approve, deny, or negotiate the amount yourself."

The strong version is not merely more detailed — it is **independently checkable**. A second reviewer (human or automated) can look at any transcript and determine, without ambiguity, whether the agent followed the rule. The weak version cannot be audited the same way, because "conservative" and "good judgment" have no fixed operational meaning across cases.

Deep Dive: Forcing Structured Output With tool_choice

```
{
  "tools": [{
    "name": "extract_invoice",
    "description": "Records the extracted fields of an invoice.",
    "input_schema": {
      "type": "object",
      "properties": {
        "invoice_number": {"type": "string"},
        "total_amount": {"type": "number"},
        "due_date": {"type": "string", "format": "date"}
      },
      "required": ["invoice_number", "total_amount"]
    }
  }],
  "tool_choice": { "type": "tool", "name": "extract_invoice" }
}
```

Forcing `tool_choice` to name `extract_invoice` directly guarantees the response is a call to that tool with schema-valid arguments — Claude cannot instead reply with a plain-text paragraph describing the invoice, and cannot return a malformed shape. What it cannot guarantee is that `total_amount` is the *correct* number if the source document was blurry, ambiguous, or genuinely missing that field — which is exactly the syntax-vs-semantics distinction from section 5.4, now visible in schema form.

Deep Dive: Recognizing a Batch-Appropriate Workload

Workload description	Batch API appropriate?
Classify 200,000 archived support tickets by category overnight	Yes — large volume, no user waiting, independent items
Live chat agent responding to a customer in real time	No — needs synchronous, low-latency, multi-turn tool calling
Nightly regeneration of product description summaries for a catalog	Yes — non-urgent, bulk, independent per-item
A CI pipeline step that must report pass/fail before merging a PR within minutes	No — needs a latency guarantee Batch does not provide

5.8 Domain 4 Exam-Ready Summary

Concept	One-line takeaway
Explicit criteria	Enumerable, checkable rules beat vague qualifiers like "be conservative"
Few-shot examples	High value for borderline/ambiguous cases; low/negative value for already-correct obvious cases
Chain-of-thought	Reserve for genuinely multi-step/judgment-heavy tasks; skip for simple lookups
Structured output	Tool-forced schemas guarantee shape/syntax, never guarantee semantic correctness
Retry loops	Fix malformed output; cannot fix answers wrong due to missing source information
Prompt caching	Use for large, stable, repeated prefixes to cut cost and latency
Batch API	~50% cheaper, asynchronous, no multi-turn tool calling — for bulk/non-urgent work only

Context Management & Reliability

Making long-running, information-heavy agents trustworthy

6.1 The Context Window as a Finite, Shared Resource

Every request shares one finite context window across the system prompt, conversation history, tool definitions, tool results, and any retrieved documents. Domain 5 treats this window as a resource an architect must actively manage, not a background detail. As a session grows — more turns, more tool calls, more retrieved content — three problems compound: **cost** rises (more tokens processed per call), **latency** rises, and **accuracy risk** rises because relevant details get diluted among a growing volume of less-relevant history.

6.2 Compaction, Summarization & the Case-Facts Pattern

When a session runs long, the standard mitigation is **compaction**: periodically summarizing older conversation history into a condensed form and replacing the verbose original turns with that summary, freeing context budget for new work while preserving the gist of what came before.

THE RISK PLAIN SUMMARIZATION INTRODUCES

Naive summarization can silently drop or blur **specific values** that matter later — an exact order number, a precise dollar amount, a specific date, a customer's exact stated constraint. A summary that says "the customer discussed a billing issue" has lost information a summary that says "customer disputes a \$214.50 charge from March 3 on order #88213" has not. This is one of the most tested failure modes in Domain 5.

THE CASE-FACTS PATTERN (HEAVILY TESTED)

Maintain a separate, structured **"case facts" block** — a running list of specific, discrete values (IDs, amounts, dates, explicit constraints, decisions already made) that is preserved verbatim and *never* subject to lossy summarization, even as the surrounding conversational narrative is compacted. When a session is summarized, the case-facts block is carried forward intact; only the prose narrative around it is condensed. This gives you the token savings of compaction without the accuracy risk of losing specific values that later steps depend on.

6.3 Lost-in-the-Middle & Positional Strategies for Long Context

Empirically, models are more reliable at attending to information placed at the **very beginning** or **very end** of a long context than information buried in the **middle** — the well-known "lost-in-the-middle" effect. This has a direct, testable architectural consequence:

POSITIONAL PLACEMENT RULE

When assembling a long context (e.g., many retrieved documents, a long research transcript, an extended conversation history), place the **most important findings, conclusions, or facts near the start** of the context (and optionally restate them near the end), rather than letting critical information sit buried in the middle of a large block of less-important material. This is why synthesis/orchestrator agents (Chapter 2) are typically instructed to lead their reports with key findings rather than building up to them narratively.

6.4 Conflicting Sources, Uncertainty & Escalation Triggers

Handling conflicting information

When two retrieved sources or two tool results disagree, the reliable behavior is **not** to silently pick one (which hides the disagreement from whoever consumes the output) and **not** to silently average or blend them (which can produce an answer neither source actually supports). The correct pattern is to **surface both, note that they conflict, and flag the topic as contested** — preserving the uncertainty rather than resolving it artificially. This directly extends the "don't invent information" principle from Chapter 1's architect mindset.

The three canonical escalation triggers

A reliable agent needs a defined policy for when to stop and hand off to a human rather than continuing autonomously. The exam recognizes three canonical triggers:

1. **Explicit human request** — the user or a policy directly asks for human review (e.g., "let me speak to a person").
2. **Policy gap** — the situation falls outside any rule the agent has been given (no explicit criterion covers this case), so proceeding would require the agent to invent a judgment call it wasn't authorized to make.
3. **Can't make progress** — the agent has attempted reasonable steps (within its tool access and retries) and is still stuck, e.g., required information is unavailable, or repeated tool failures block further action (tying back to the circuit-breaker pattern in 2.7).

DISTRACTOR PATTERN

An option describing an agent that "makes its best guess and proceeds" when facing a genuine policy gap or missing information is a distractor whenever a human-escalation option is also available — reliable systems escalate uncertainty rather than resolving it by fabrication, which is the same principle from Chapter 1 restated at the reliability layer.

6.5 Reliability Engineering: Verification Loops & Grounding

Reliability in a context-management sense means the agent's stated conclusions are actually **grounded** in the context it was given, rather than drawn from the model's general training knowledge when the task specifically calls for using provided documents/data. Practical grounding techniques tested on the exam:

- **Require citation of source spans** for factual claims drawn from retrieved documents, so a downstream check (automated or human) can verify the claim traces back to actual provided content rather than to the model's own prior knowledge.
- **Separate "what the documents say" from "what I recommend"** explicitly in the output structure, so a reviewer can tell which parts are extracted fact versus model judgment.
- **Pair with the verification phase of the agentic loop (2.1)** — an agent that claims to have retrieved information should have that claim checkable against the actual tool_result content, not just trusted at face value.

6.6 Deep Dive: Compaction With a Preserved Case-Facts Block

Section 6.2 introduced the case-facts pattern conceptually. Here is what it looks like applied to a long-running support conversation just before and after a compaction pass.

NAIVE COMPACTION — LOSES SPECIFICITY

"Summary: Customer had a billing dispute that was eventually resolved after some back-and-forth about the amount and timing."

CASE-FACTS-PRESERVING COMPACTION

```
CASE_FACTS: {  
  "order_id": "88213",  
  "disputed_amount": "$214.50",  
  "charge_date": "2026-03-03",  
  "resolution": "partial_refund_approved",  
  "refund_amount": "$107.25"  
}  
NARRATIVE_SUMMARY: "Customer disputed a charge as duplicate; investigation  
confirmed a partial duplicate; a 50% partial refund was approved  
and communicated."
```

If a fourth agent later re-opens this case, the naive summary gives them nothing to act on precisely — they would have to ask the customer to repeat the order ID and amount. The case-facts version gives any downstream step (a new agent, a human reviewer, an audit tool) exact values it can act on immediately, while the narrative portion still gets the token-savings benefit of compaction.

Deep Dive: Applying the Lost-in-the-Middle Rule to a Research Report

Key finding first

stated in first paragraph

VS.

Key finding buried

revealed only in paragraph 9

A synthesis agent producing a ten-paragraph competitive analysis should open with "Competitor B has priced 40% below market and is the primary threat" rather than building toward that conclusion narratively across nine paragraphs of background — not for stylistic reasons, but because any downstream consumer (human or another agent reading this report as input) is statistically less reliable at extracting a fact it must search for mid-document than one stated up front.

6.7 Retrieval Strategies for Grounded Context

Many context-management scenarios involve retrieval-augmented generation (RAG): rather than stuffing an entire document corpus into context, relevant chunks are retrieved on demand (typically via semantic search over embeddings) and injected into the prompt only when needed. Domain 5 tests the architectural judgment around this pattern, not embedding-model internals (which are explicitly out of scope, section 8.4).

- **Chunking granularity is a trade-off.** Chunks too large waste context budget on irrelevant surrounding text and dilute the positional advantage from 6.3; chunks too small can separate a fact from the context needed to correctly interpret it (e.g., splitting a number from the clause that defines its units).

- **Retrieval quality bounds generation quality.** If the retrieval step fails to surface the chunk containing the actual answer, no amount of prompt engineering or retrying (5.5) can produce a correct response — this is the same "fix upstream, not by retrying" principle from Domain 4, now specifically located at the retrieval stage.
- **Retrieved content should be attributable.** Each retrieved chunk should carry enough source metadata (document name, section, date) that the grounding techniques from 6.5 — citing source spans, distinguishing extracted fact from model judgment — are actually possible to implement.

DISTRACTOR PATTERN

A scenario where an agent gives a wrong answer because the retrieval step missed the relevant chunk, paired with an option suggesting "ask Claude to think more carefully" or "add a retry," is testing whether you correctly locate the defect in retrieval rather than generation — no amount of downstream prompting fixes a document that was never retrieved into context in the first place.

6.8 Domain 5 Exam-Ready Summary

Concept	One-line takeaway
Context window	A finite, shared resource across prompt, history, tools, and retrieved content — manage it actively
Compaction	Summarize old turns to save tokens, but never let summarization touch specific values
Case-facts block	Preserve exact IDs/amounts/dates/decisions verbatim, separate from the summarized narrative
Lost-in-the-middle	Place key findings at the start (and optionally the end) of long context, not buried mid-block
Conflicting sources	Surface both and flag as contested — never silently pick one or blend them
Escalation triggers	Explicit human request, policy gap, or inability to make progress — always escalate, never guess
Grounding	Require traceable citations; separate extracted fact from model judgment

The Six Official Scenario Contexts

Every real exam question is drawn from 4 of these 6 contexts — study each lens

Anthropic draws every CCAR-F question from one of six recurring scenario contexts (4 of the 6 appear in any given sitting, randomized). You will not be tested on "list the six contexts" — you will be tested on applying Chapters 2–6 *inside* one of these settings. This chapter walks each context end-to-end so the domain concepts you just studied become instantly recognizable no matter which contexts appear on your exam.

7.1 Customer Support Resolution Agent

A conversational agent that resolves customer issues (order status, refunds, account changes) using tools that query order systems, issue refunds, and update account records.



This scenario is a single-agent, tool-heavy loop rather than a multi-agent system — its architectural challenges are almost entirely about the *quality and safety* of individual tool calls rather than coordination between agents, which is why Domains 1, 2, and 4 dominate its question surface.

Domain	How it shows up here
Agentic loop (D1)	Gather context = look up the order; act = issue refund or update; verify = confirm the refund actually posted before telling the customer it's done
Hooks (D1)	PreToolUse hook blocking any refund above a dollar threshold without human approval — a textbook "costly, irreversible action needs a deterministic gate" case
Tool design (D2)	<code>is_error: false</code> for "no order found for that ID" (valid outcome) vs. <code>true</code> for a downstream order-system outage
Prompt engineering (D4)	Explicit categorical refund-approval criteria (amount, days-since-purchase, prior-refund count) instead of "use good judgment"
Context (D5)	Case-facts block preserving the exact order ID and disputed amount across a long, multi-turn troubleshooting conversation

7.2 Code Generation with Claude Code

A developer uses Claude Code inside a real repository to implement a feature, fix a bug, or refactor code, relying on CLAUDE.md, project configuration, and (optionally) subagents.

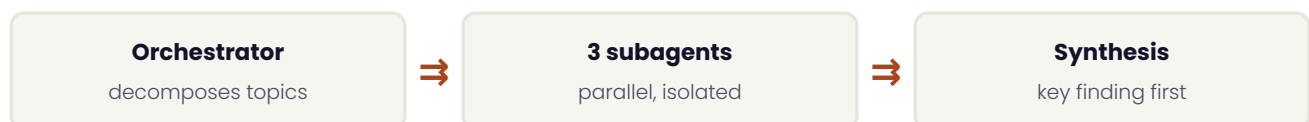


The most commonly missed detail in this scenario is the verification step: an agent that stops as soon as code *compiles*, without confirming the actual test suite passes, has skipped the "verify" phase of the agentic loop from section 2.1 — a subtle but frequently tested failure.

Domain	How it shows up here
Claude Code config (D3)	CLAUDE.md documents build/test commands and conventions; project-level <code>.claude/</code> and <code>.mcp.json</code> settings are shared via git so every teammate gets the same setup
Permissions (D3)	<code>allowedTools</code> restricting shell access to safe, expected commands (test runners, linters) rather than arbitrary shell execution
Subagents (D1/D3)	A subagent with <code>context:fork</code> runs the noisy full test suite and reports back only pass/fail plus failures — keeping the main session's context clean
Verification (D1)	The loop isn't "done" when code compiles — it's done once tests actually pass, which the agent must check, not assume

7.3 Multi-Agent Research System

An orchestrator decomposes a broad research question into topic-specific sub-tasks, dispatches subagents to investigate each, and synthesizes their findings into a report.

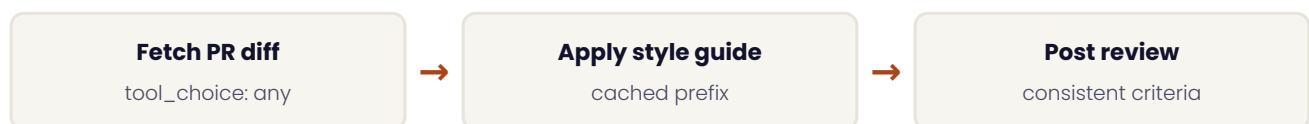


This is the canonical scenario for Domain 1's hub-and-spoke, decomposition, and context-isolation content simultaneously — expect it to be paired with Domain 5's lost-in-the-middle and conflicting-sources content in the same question, since the synthesis step is exactly where those two domains intersect.

Domain	How it shows up here
Decomposition (D1)	Each subagent investigates one independent sub-topic; a topic missing entirely from the final report traces back to the orchestrator's planning step, not to any individual subagent
Topology (D1)	Hub-and-spoke: orchestrator dispatches to N independent research subagents, none of which need to talk to each other
Context isolation (D1)	Each subagent's raw search results and dead-end explorations stay inside its own context; only its synthesized findings return to the orchestrator
Tool scoping (D2)	Research subagents get search/fetch tools only — no write or transactional tools, since their job is purely informational
Lost-in-the-middle (D5)	The final report leads with the most important findings rather than building up to them at the end
Conflicting sources (D5)	Two subagents returning contradictory facts about the same topic must both be surfaced and flagged, not silently reconciled

7.4 Developer Productivity with Claude

Broader day-to-day developer use of Claude (API and/or Claude Code) for tasks like code review, documentation generation, or answering questions about a codebase — less about one big autonomous task, more about many smaller, well-scoped interactions.



Unlike 7.2's autonomous code-generation loop, this scenario is characterized by many short, independent interactions rather than one long-running task — which is exactly the profile where prompt caching (5.6) on a large, stable style guide delivers the most value, since the same expensive prefix is reused across every single review.

Domain	How it shows up here
tool_choice (D2)	A code-review assistant that always needs to fetch the diff uses <code>tool_choice: any</code> ; a general Q&A assistant that only sometimes needs a tool uses <code>auto</code>
Few-shot examples (D4)	Useful for calibrating review tone/strictness on ambiguous style questions; unnecessary for clear-cut lint violations the model already flags correctly
Prompt caching (D4)	A large, stable style guide reused across many review requests is a strong prompt-caching candidate
CLAUDE.md (D3)	Encodes the team's review standards once so every review session applies them consistently

7.5 Claude Code for CI/CD

Claude Code embedded in an automated pipeline — for example, triggered on every pull request to run checks, generate summaries, or attempt automated fixes — with no human present to interactively approve each step.



Of all six scenarios, this one carries the highest concentration of Domain 1's deterministic-control principle and Domain 3's headless-mode mechanics simultaneously — no human is present to notice a runaway loop or approve a risky action, so every safeguard that would otherwise be "nice to have" (timeouts, iteration caps, scoped allow-lists) becomes structurally mandatory.

Domain	How it shows up here
Headless mode (D3)	Triggered via the <code>-p</code> flag from the CI system; must run start-to-finish without interactive prompts
Scoped permissions (D2/D3)	An unattended pipeline is the highest-stakes case for least-privilege — allow only the exact commands the pipeline step needs, deny everything else by default
Deterministic gates (D1)	Any action beyond "comment on the PR" (e.g., auto-merging, auto-deploying) needs a hook-level or pipeline-level approval gate, not a prompt instruction to "be careful"
Termination controls (D1)	A hard timeout and iteration cap are mandatory in CI, where no human is watching to notice a stuck loop
Machine-checkable output (D3/D4)	The pipeline branches on an exit code or structured result, not on parsing prose

7.6 Structured Data Extraction

Claude extracts structured fields (e.g., invoice data, form fields, entities from unstructured text) from documents at scale, typically feeding a downstream system that expects a strict schema.



The exam distinguishes this scenario from 7.5 by asking about semantic correctness rather than execution safety: the risk here isn't a runaway action, it's a syntactically perfect but factually wrong or fabricated field value — which is why the shape-vs-semantics distinction from section 5.4 is this scenario's defining concept.

Domain	How it shows up here
Tool-forced structured output (D4)	Define the target schema as a tool's <code>input_schema</code> and force it via <code>tool_choice</code> , guaranteeing shape — but not guaranteeing every extracted value is semantically correct
Validation/retry limits (D4)	A retry loop fixes malformed JSON; it cannot invent a field's value if that value was never present in the source document — that failure needs better source data or an explicit "field not found" output, not another retry
Escalation on missing data (D5)	When a required field genuinely isn't in the document, the reliable behavior is to return an explicit null/unknown marker and (if configured) escalate — not to fabricate a plausible-looking value
Batch API fit (D4)	Large-volume, non-urgent extraction over a document backlog is a strong Batch API candidate — 50% cheaper, and the asynchronous latency is acceptable for a bulk job

Final Review System

Compression pass — no new concepts, pure recall reinforcement

8.1 The Distractor–Recognition Cribsheet

Across all five domains, wrong answers on this exam are rarely random — they follow a small number of recurring patterns. Recognizing the *pattern* of a distractor is often faster than re-deriving the right answer from scratch.

Distractor pattern	Where it appears	The tell
Treating <code>tool_use</code> as a final answer	D1, D2	Any option that returns text to the user instead of executing the tool and continuing the loop
Prompt instruction offered for a costly/irreversible failure	D1, D3, D4	"Add a clearer instruction," "tell Claude to be careful," "strengthen the system prompt" — wrong whenever a hook/permission alternative exists
Blaming a subagent for an orchestrator-level gap	D1	Missing topic/coverage in a final result — diagnose the coordinator's decomposition first
Personal-scope config for a team-shared need	D3	"Every team member should get this" answered with <code>~/.claude/</code> or <code>~/.claude.json</code> instead of the project-level equivalent
Forcing tool use every turn	D2	<code>tool_choice: any</code> proposed for a turn that doesn't structurally guarantee action is needed
Confusing shape-guarantee with correctness-guarantee	D4	Claiming tool-forced JSON "ensures the data is right," not just "ensures the JSON is valid"
Retrying to fix missing information	D4, D5	Suggesting another retry/validation pass when the real defect is that the source data was never provided
Batch API for a live, interactive workload	D4	Any user-facing, time-sensitive scenario paired with a Batch API answer
Silently resolving conflicting information	D5	Picking one source, or averaging two sources, instead of surfacing both and flagging the conflict
Guessing instead of escalating	D1, D5	"Makes its best guess and proceeds" when a policy gap or missing-information scenario is described

8.2 Master Decision-Frameworks Index

These are the core "if X, then Y" frameworks this guide builds — review each one until you can restate it without looking.

1. THE PROMPTS-VS-HOOKS RULE (CHAPTER 1.5)

Single costly/irreversible failure possible → deterministic control (hook/permission). Quality/tone/style preference → prompt instruction is sufficient.

2. DECOMPOSITION FITNESS (2.2)

Independent, separately-verifiable sub-tasks → decompose. Tightly coupled, constant shared context needed → keep as one agent.

3. TOPOLOGY SELECTION (2.4)

Independent parallel sub-tasks → hub-and-spoke (default). Sequential staged processing → pipeline. Genuine peer negotiation → peer-to-peer (rare).

4. ISERROR SEMANTICS (3.3)

Tool executed successfully, zero results → `is_error: false`. Tool execution itself failed → `is_error: true`.

5. TOOL_CHOICE SELECTION (3.2)

Action not guaranteed every turn → `auto`. Action guaranteed but tool varies → `any`. Need one specific schema-shaped output → named tool.

6. CONFIG SCOPE PLACEMENT (4.2 / 4.3)

Must be identical for every team member / travels with the repo → project scope (`.claude/`, `.mcp.json`). Personal, machine-specific, cross-project → user scope (`~/.claude/`, `~/.claude.json`).

7. FEW-SHOT TARGETING (5.2)

Borderline/ambiguous case the model struggles with → add a few-shot example of that exact case. Already-correct obvious case → skip; examples add no value.

8. DIAGNOSING A WRONG STRUCTURED-OUTPUT VALUE (5.4 / 5.5)

Wrong shape/syntax → retry/validation loop fixes it. Wrong content because the fact wasn't in the source → fix upstream context/retrieval; retrying will not help.

9. BATCH VS. STANDARD API (5.6)

Large volume, non-urgent, independent items → Batch API. Live, interactive, multi-turn tool-calling → standard API.

10. CONTEXT PLACEMENT UNDER LOST-IN-THE-MIDDLE (6.3)

Most important finding/conclusion → place at the start of the context (optionally restate at the end). Supporting detail → the middle is acceptable.

11. ESCALATION TRIGGERS (6.4)

Explicit human request, OR policy gap, OR agent cannot make further progress → escalate to a human. None of these true → continue autonomously within existing rules.

8.3 Complete Glossary of Exam Terminology

Term	Definition
Agentic loop	The cycle of gathering context, taking action via tools, verifying the result, and repeating until the goal or a boundary is reached
stop_reason	Field on a Messages API response indicating why generation stopped (<code>end_turn</code> , <code>tool_use</code> , <code>max_tokens</code> , <code>stop_sequence</code> , <code>pause_turn</code> , <code>refusal</code>)
Orchestrator / coordinator	The agent responsible for decomposing a goal and dispatching sub-tasks to subagents, then synthesizing their results
Subagent	An agent executing one bounded sub-task in its own isolated context window
Context isolation	Keeping a subagent's internal working context separate from the orchestrator/parent, so only a final summary crosses back
Hub-and-spoke	Multi-agent topology where a central orchestrator dispatches to independent subagents that don't communicate with each other
Pipeline topology	Multi-agent topology where agents are chained sequentially, each stage's output feeding the next
Hook	Host-enforced code that runs at a defined point in the agentic loop (e.g., PreToolUse, PostToolUse), independent of model behavior
PreToolUse / PostToolUse	Hook points firing immediately before / after a tool call executes
Idempotency key	A unique identifier per logical operation that lets a retried, state-changing tool call be safely deduplicated
Circuit breaker	A control that halts retries after repeated failures of the same action, escalating instead of looping indefinitely
tool_choice	Messages API parameter controlling whether/which tool Claude must call: <code>auto</code> , <code>any</code> , a named tool, or <code>none</code>
is_error	Flag on a tool_result indicating whether the tool call itself failed (true) versus succeeded with any outcome including zero results (false)
MCP (Model Context Protocol)	Open protocol standardizing how AI applications connect to external tools, resources, and prompts across hosts
Host / Client / Server (MCP)	Host = user-facing app; Client = protocol connector inside the host; Server = process exposing tools/resources/prompts
MCP Resource	Application-controlled, addressable data a host can read into context

MCP Prompt	A reusable, server-defined prompt template surfaced by the host (e.g., as a slash command)
stdio transport	MCP transport where the server runs as a local subprocess communicating over standard input/output
Least privilege (tool scoping)	Exposing each agent only the small set of tools relevant to its task, reducing both selection errors and blast radius
CLAUDE.md	Markdown file automatically loaded by Claude Code to give an agent durable, project-specific memory
.mcp.json	Project-root file registering MCP servers shared with the whole team via git
~/.claude.json / ~/.claude/	User-home-directory config for personal, cross-project settings never committed to git
allowedTools	Claude Code settings.json permission list constraining which tools/commands an agent may execute
context: fork	Configuration that runs a subagent/skill in an isolated context window, returning only its summarized result to the parent
Headless mode / -p flag	Non-interactive Claude Code invocation used to embed it in scripts and CI/CD pipelines
Claude Agent SDK	Framework providing the agentic loop, tool execution, permissions, and subagent scaffolding for building custom agent applications
Explicit categorical criteria	Enumerable, checkable rules in a prompt, preferred over vague qualifiers like "be conservative"
Few-shot examples	Worked input/output examples in a prompt, most valuable for borderline/ambiguous cases
Extended thinking / chain-of-thought	A dedicated reasoning phase before the final answer, reserved for genuinely multi-step or judgment-heavy tasks
Tool-forced structured output	Using a schema-defined tool plus a forced tool_choice to constrain output shape at generation time
Prompt caching	Reusing a cached, unchanging prompt prefix across requests to cut cost and latency
Batch API	Asynchronous, ~50%-discounted API mode with no real-time SLA and no multi-turn tool calling, for bulk/non-urgent work
Compaction	

	Summarizing older conversation history to free context budget in a long-running session
Case-facts block	A structured record of exact values (IDs, amounts, dates, decisions) preserved verbatim through compaction
Lost-in-the-middle	The tendency for models to attend less reliably to information buried mid-context versus at the start/end
Grounding	Ensuring stated conclusions trace back to actual provided context/data rather than unsupported model assertion
Escalation trigger	One of three conditions (explicit human request, policy gap, cannot make progress) requiring handoff to a human

8.4 Out-of-Scope Topics & Exam-Day Logistics

CONFIRMED OUT OF SCOPE — DO NOT SPEND STUDY TIME HERE

Constitutional AI and RLHF theory; model fine-tuning; embedding models and vector-database internals; the Computer Use and Vision APIs; low-level streaming/SSE mechanics; OAuth and API-key rotation internals; rate-limit and quota arithmetic; exact token-pricing memorization; cloud-provider-specific deployment configuration.

EXAM-DAY LOGISTICS CHECKLIST

- The exam is **proctored and cannot be paused** once started — block a genuinely free 2.5-hour window, not just the 120-minute exam length, to account for check-in and environment verification.
- Ensure your test environment meets the proctoring requirements (working webcam and microphone, clean desk, single monitor, no unauthorized materials or applications running).
- Closed-book, no outside AI assistance — this guide (and your own practice) is your preparation, not a reference during the exam.
- There is **no penalty for an incorrect answer** — always select your best answer rather than leaving any of the 60 questions blank.
- Scoring is scaled (100–1000, pass at 720), not a raw percentage — don't try to reverse-engineer a "number of questions you can miss" from a percentage assumption.

8.5 Final 24-Hour Review Checklist

In your last day before the exam, do not re-read this guide start to finish. Instead, work through this checklist and only re-open a chapter section when you genuinely cannot complete an item from memory.

1. Recite the agentic loop (gather context → act → verify → repeat) and all six `stop_reason` values with their correct handling, unaided.
2. State the prompts-vs-hooks rule and give one irreversible-action example that requires a hook.
3. Explain why a missing topic in a research report points to the orchestrator, not a subagent.
4. Name all three multi-agent topologies and the default choice when a scenario doesn't clearly signal sequential dependency.
5. State the correct `is_error` value for "zero results, successful call" versus "call failed."
6. State the difference between `.mcp.json` and `~/.claude.json`, and the general path-prefix rule for all Claude Code config scope.
7. Explain what the `-p` flag enables and why it matters for CI/CD.
8. Explain why tool-forced structured output guarantees shape but not semantic correctness.
9. State when a retry loop can fix a bad structured-output value and when it cannot.
10. State the Batch API's three defining constraints (cost, latency/SLA, multi-turn tool calling) and when it fits.
11. Explain the case-facts pattern and why plain summarization is risky without it.
12. Name the three escalation triggers from memory.

YOU ARE READY WHEN...

...you can complete every item above without flipping back to a chapter, and you can look at any of the ten patterns in the distractor-recognition cribsheet (8.1) and immediately name which domain concept resolves it. At that point, you have covered 100% of the CCAR-F syllabus in proportion to how it is actually weighted and scored — go into the exam with confidence.

Complete Technical Reference

Consolidated lookup tables spanning every domain — use for quick recall, not first-time learning

This appendix consolidates the technical reference material scattered across Chapters 1–8 into single lookup tables. Use it in the final week of preparation to spot-check recall, and during your last review pass (section 8.5) as a fast cross-reference — every fact here was already taught in context earlier in this guide.

9.1 Messages API Parameter Reference

Parameter	Location	Purpose	Architect's note
<code>model</code>	Top-level	Selects which Claude model handles the request	Different tasks in one system can route to different models by cost/capability need
<code>max_tokens</code>	Top-level	Caps the length of the generated response	Set too low, causes a <code>max_tokens</code> <code>stop_reason</code> mid-answer/mid-JSON
<code>system</code>	Top-level	Durable role/instruction context, separate from the message list	Best location for explicit categorical criteria (5.1) and stable, cacheable content (5.6)
<code>messages</code>	Top-level	Ordered array of user/assistant turns, including <code>tool_use</code> and <code>tool_result</code> blocks	The entire "state" of a session lives here; compaction (6.2) operates on this array
<code>tools</code>	Top-level	Declares available tools (name, description, <code>input_schema</code>)	Description quality directly drives tool-selection accuracy (3.1)
<code>tool_choice</code>	Top-level	Constrains whether/which tool must be called this turn	See the full decision table in section 3.2
<code>stop_sequences</code>	Top-level	Custom strings that force generation to halt when produced	Useful for constraining free-text generation to a known delimiter
<code>stop_reason</code>	Response field	Explains why generation stopped	The single most exam-relevant field in the entire API – see 1.3

9.2 stop_reason Quick Reference

Value	Terminal or mid-loop?	Correct next action
end_turn	Terminal	Return result to caller; loop ends
tool_use	Mid-loop	Execute tool(s), append tool_result, re-send
max_tokens	Terminal but incomplete	Do not treat as a finished answer; raise budget or reduce scope and retry
stop_sequence	Terminal (intentional)	Parse up to the matched sequence
pause_turn	Mid-loop (resumable)	Resume by sending the paused turn back, not by starting over
refusal	Terminal	Do not blindly retry the identical request; escalate or reroute

9.3 Claude Code Configuration File Reference

File	Scope	Git-tracked?	Holds
<code>CLAUDE.md</code> (repo root)	Project	Yes	Project-wide agent memory: commands, conventions, structure, gotchas
<code>CLAUDE.md</code> (subdirectory)	Project (module-scoped)	Yes	Module-specific memory, loaded only when working in that area
<code>.mcp.json</code>	Project	Yes	MCP servers shared with every contributor to this repo
<code>.claude/settings.json</code>	Project	Yes	Shared permissions (<code>allowedTools</code> / deny-lists) and hooks
<code>.claude/commands/*.md</code>	Project	Yes	Shared slash commands
<code>~/.claude/settings.json</code>	User/personal	No	Personal permission overrides, individual preferences
<code>~/.claude.json</code>	User/personal	No	Personal MCP servers, applied across all of that user's projects
Enterprise/managed policy	Organization	N/A	Mandatory security policy that cannot be overridden by user or project settings

9.4 Hook Events Reference

Hook	Fires	Can it block the action?	Typical use
<code>PreToolUse</code>	Before a tool call executes	Yes	Validation, blocking disallowed calls, requiring approval
<code>PostToolUse</code>	After a tool call returns	No (action already happened)	Logging, audit trail, automated verification, sanitizing output

WHY THE DISTINCTION MATTERS

Only a **PreToolUse** hook can *prevent* a harmful action from occurring at all. A **PostToolUse** hook can only react after the fact — valuable for audit and verification, but not a substitute for a blocking pre-check when the action itself must never be allowed to happen under certain conditions.

9.5 MCP Primitives & Transport Reference

Element	Category	Invoked by	Notes
Tool	Primitive	The model, autonomously, mid-loop	Same selection/description principles as native tools (3.1–3.3)
Resource	Primitive	The host application	Application-controlled data surfaced into context, not autonomously fetched by the model the way a tool is
Prompt	Primitive	The user, via the host's UI (e.g., a slash command)	Reusable, parameterized templates for common workflows
stdio	Transport	—	Local subprocess; simplest; ideal for local/filesystem-adjacent tools
HTTP-based	Transport	—	Remote/shared server; needed when many hosts/users share one server instance

9.6 Multi-Agent Topology Reference

Topology	Communication pattern	Default?	Best-fit signal in a scenario
Hub-and-spoke	Central orchestrator ↔ each subagent; no subagent-to-subagent link	Yes, default	Independent, parallelizable sub-tasks
Pipeline	Sequential chain, each stage feeds the next	No	Explicit sequential/staged processing language ("first extract, then validate, then load")
Peer-to-peer	Agents communicate directly, no central coordinator	No — rarely correct	Genuine negotiation between equally-privileged agents; hardest to observe/bound

9.7 Cost & Latency Optimization Reference

Technique	Optimizes	Trade-off / constraint
Prompt caching	Cost + latency on repeated, stable prompt prefixes	Only helps when a large prefix is genuinely reused across many calls unchanged
Batch API	Cost (~50% discount)	Asynchronous, no real-time SLA, no multi-turn tool calling – bulk/non-urgent workloads only
Tool scoping (least privilege)	Accuracy (fewer wrong tool picks) and security (blast radius)	Requires maintaining per-agent tool subsets rather than one universal list
Context compaction	Cost + accuracy in long sessions	Must preserve a case-facts block to avoid losing specific values (6.2)
context: fork / subagent isolation	Keeps the parent session's context small and clean	Only the subagent's final summary returns – requires a good result contract (2.8)

9.8 Cross-Domain Synthesis: How the Five Domains Reinforce Each Other

By this point you have studied each domain in isolation. On the actual exam, questions frequently sit at the intersection of two or three domains at once, so it is worth explicitly naming how they connect:

- **Domain 1 ↔ Domain 2:** Good orchestration is worthless if the tools an agent (or subagent) calls are poorly described or over-broad – decomposition and topology decisions (D1) only pay off when paired with tightly-scoped, well-described tools (D2).
- **Domain 1 ↔ Domain 3:** Hooks and permission boundaries (D1's deterministic-control principle) are *implemented* concretely through Claude Code's `settings.json` and hook configuration (D3) – D1 is the theory, D3 is the product-level mechanism.
- **Domain 2 ↔ Domain 4:** Tool-forced structured output (D4) is literally an application of tool-calling mechanics (D2, especially `tool_choice`) repurposed for schema enforcement rather than real-world action.
- **Domain 4 ↔ Domain 5:** Diagnosing whether a bad output needs a retry (D4) or better context/retrieval (D5) is the same underlying judgment call – is the defect in the model's generation, or in what it was given to work with?

- **Domain 1 ↔ Domain 5:** Escalation triggers (D5) are the context-and-reliability-layer expression of the same "don't guess, don't fabricate" principle that governs verification in the agentic loop (D1) and result contracts between orchestrators and subagents (D1).

FINAL FRAMING BEFORE YOU SIT THE EXAM

If a question feels like it belongs to more than one domain, that is not a sign you have misread it — it is a sign the exam is testing exactly this kind of cross-domain synthesis. Identify every constraint the scenario states, match each one to the concept that resolves it (using the decision-framework index in 8.2 if needed), and choose the option that satisfies all of them simultaneously rather than the option that only addresses the most obvious one.

Named Architecture Patterns Reference

Fifteen recurring design patterns, each mapped to the domain and section that teaches it

Production Claude systems are built from a small, recurring set of named patterns. Recognizing which pattern a scenario is describing — often before you've fully read every option — is one of the fastest paths to the correct answer. Each entry below states the problem it solves, the mechanism, and where in this guide it was taught in full.

1. The Verified Action Pattern

Problem: An agent reports a task as complete when the underlying action didn't actually succeed. **Mechanism:** After every state-changing tool call, the surrounding code (or a follow-up tool call) independently confirms the resulting state matches the intended outcome before the loop advances. **Taught in:** 2.1.

2. The Deterministic Gate Pattern

Problem: A costly or irreversible action must never occur under disallowed conditions, and a prompt instruction alone cannot provide that guarantee. **Mechanism:** A PreToolUse hook or permission rule intercepts the call in ordinary application code and blocks it outright when conditions aren't met. **Taught in:** 1.5, 2.5, 4.4.

3. The Isolated Subagent Pattern

Problem: A sub-task generates large volumes of noisy intermediate work (search attempts, failed calls, exploratory reasoning) that would pollute a shared context window. **Mechanism:** Run the sub-task in its own agent with its own context window; return only a final, structured summary to the parent. **Taught in:** 2.3, 4.5 (context:fork).

4. The Structured Result Contract Pattern

Problem: A coordinator cannot make good decisions about a subagent's work if it cannot tell partial success from full success. **Mechanism:** Require every subagent to return a fixed schema including an explicit status field and an "unresolved" list, rather than free-text prose. **Taught in:** 2.3, 2.8.

5. The Idempotency Key Pattern

Problem: A retried tool call against a non-idempotent action would duplicate its real-world effect. **Mechanism:** Generate one unique key per logical operation; the receiving system deduplicates retries carrying the same key. **Taught in:** 2.6.

6. The Circuit Breaker Pattern

Problem: An agent stuck in a failing action retries indefinitely, burning cost and time.

Mechanism: Track consecutive failures of the same action; once a threshold is hit, stop retrying and escalate instead. **Taught in:** 2.6, 2.7, 6.4.

7. The Least-Privilege Tool Scope Pattern

Problem: An agent with access to every tool in the system has a larger blast radius and worse tool-selection accuracy than it needs. **Mechanism:** Expose each agent/subagent only the small set of tools (~4-5) directly relevant to its bounded task. **Taught in:** 3.6.

8. The Routing-Grade Description Pattern

Problem: Vague tool descriptions cause wrong-tool selection or malformed parameters.

Mechanism: Write tool and parameter descriptions as carefully as prompts: what it does, when to use it, parameter formats, and what it returns. **Taught in:** 3.1, 3.7.

9. The Explicit Categorical Criteria Pattern

Problem: Vague qualifiers ("be conservative") produce inconsistent behavior across similar cases. **Mechanism:** State enumerable, checkable rules with concrete thresholds instead of subjective language. **Taught in:** 5.1, 5.7.

10. The Schema-Forced Output Pattern

Problem: Prompt-only JSON instructions can still produce malformed output. **Mechanism:** Define the desired shape as a tool's input_schema and force that tool via tool_choice, guaranteeing shape (not semantic correctness). **Taught in:** 3.2, 5.4, 5.7.

11. The Case-Facts Preservation Pattern

Problem: Summarizing long conversation history for context savings risks losing specific values (IDs, amounts, dates) needed later. **Mechanism:** Maintain a separate structured block of exact values that is never subject to lossy summarization, alongside a compacted narrative. **Taught in:** 6.2, 6.6.

12. The Lead-With-Findings Pattern

Problem: Important conclusions buried mid-document are less reliably picked up due to the lost-in-the-middle effect. **Mechanism:** Place the most important finding at the start of a long context or report, optionally restating it at the end. **Taught in:** 6.3, 6.6.

13. The Flag-Don't-Resolve Pattern

Problem: Two sources disagree, and silently picking one (or blending them) hides real uncertainty. **Mechanism:** Surface both positions explicitly and mark the topic as contested rather than resolving it artificially. **Taught in:** 6.4.

14. The Escalate-on-Boundary Pattern

Problem: An agent facing missing policy coverage or an explicit human request continues autonomously anyway. **Mechanism:** Recognize the three canonical triggers (explicit request, policy gap, cannot make progress) and hand off to a human rather than guessing. **Taught in:** 6.4.

15. The Scoped Headless Automation Pattern

Problem: An unattended CI/CD-triggered agent has no human present to catch a mistake or approve a risky action. **Mechanism:** Combine headless (`-p`) invocation with a tightly scoped allow-list, hard timeouts/iteration caps, and machine-checkable (exit-code/structured) output. **Taught in:** 4.6, 7.5.

Recognized Failure Patterns Reference

The mirror image of Chapter 10 — twelve ways these systems break, and why

Just as the exam rewards recognizing named patterns, it rewards recognizing named *anti-patterns* — recurring, specifically wrong designs that show up as plausible-looking distractors. Each entry names the anti-pattern, describes the symptom a scenario uses to signal it, and states the correct pattern that replaces it.

Anti-pattern	Symptom in the scenario	Correct replacement
Trust-the-claim completion	Agent declares success without checking real-world state	Verified Action Pattern (Ch.10 #1)
Prompt-only risk control	A costly/irreversible failure is guarded only by wording in the system prompt	Deterministic Gate Pattern (Ch.10 #2)
Leaky subagent context	Orchestrator's context fills with a subagent's raw exploratory work	Isolated Subagent Pattern (Ch.10 #3)
Silent partial failure	A subagent's incomplete work looks identical to full success at the orchestrator level	Structured Result Contract Pattern (Ch.10 #4)
Unsafe blind retry	A payment/notification/state-changing call is retried with no deduplication	Idempotency Key Pattern (Ch.10 #5)
Infinite retry loop	The same failing action is attempted indefinitely with no cap	Circuit Breaker Pattern (Ch.10 #6)
Universal tool exposure	A single agent has access to every tool in the system regardless of task	Least-Privilege Tool Scope Pattern (Ch.10 #7)
Minimal tool description	A tool description reads like a one-line label ("gets data") with no usage guidance	Routing-Grade Description Pattern (Ch.10 #8)
Vague qualifier instruction	Prompt says "be conservative" / "use good judgment" for a decision needing consistency	Explicit Categorical Criteria Pattern (Ch.10 #9)
Prompt-only JSON request	Output format is requested purely via natural-language instruction with no schema enforcement	Schema-Forced Output Pattern (Ch.10 #10)
Lossy narrative compaction	A long session summary drops exact IDs, amounts, or dates needed downstream	Case-Facts Preservation Pattern (Ch.10 #11)
Buried-conclusion reporting	The most important finding appears only deep in the middle of a long report	Lead-With-Findings Pattern (Ch.10 #12)

ADDITIONAL JUDGMENT-BASED ANTI-PATTERNS (NO SINGLE FIX — DIAGNOSE FIRST)

- **Retrying to manufacture missing facts:** re-invoking the model hoping a different answer appears, when the real defect is that the required information was never in the source context (5.5). Fix upstream, not by retrying generation.
- **Batch API for live workloads:** choosing the cheaper asynchronous API for a scenario that structurally requires synchronous, low-latency, multi-turn interaction (5.6). Cost savings never override a hard latency/interactivity requirement.
- **Guessing under uncertainty:** an agent resolving a policy gap or missing data by fabricating a plausible value instead of escalating (6.4). Preserving uncertainty is always preferred to inventing certainty.

HOW TO USE CHAPTERS 10–11 TOGETHER

When a scenario describes a failure, first search Chapter 11 for the symptom that matches, then confirm the fix against the corresponding pattern in Chapter 10, then verify your answer choice implements that exact mechanism — not a superficially similar one (e.g., a prompt-based fix offered where the anti-pattern specifically calls for a deterministic one).

One Card Per Domain

Flip through these in the final minutes before your exam — each stands alone

Each of the following five cards distills its entire domain to the smallest set of statements that, if internalized, resolve the majority of that domain's scenario questions. Read a card, close your eyes, and try to reconstruct it before turning the page.

Agentic Architecture & Orchestration

IF YOU REMEMBER ONLY FIVE THINGS

1. `tool_use` is always mid-loop — execute the tool and continue, never treat it as a final answer.
2. A single costly/irreversible failure demands a hook or permission gate, never a prompt instruction alone.
3. Missing coverage in a multi-agent result traces to the orchestrator's decomposition, not to any one subagent.
4. Default to hub-and-spoke topology unless the scenario clearly signals sequential stages or genuine peer negotiation.
5. Every loop needs a host-enforced termination condition: iteration cap, budget, timeout, or failure circuit breaker.

KEY TERMS TO RECALL UNAIDED

stop_reason end_turn tool_use pause_turn refusal orchestrator subagent
context isolation hub-and-spoke pipeline PreToolUse PostToolUse idempotency key
circuit breaker

Tool Design & MCP Integration

IF YOU REMEMBER ONLY FIVE THINGS

1. Tool descriptions are the routing mechanism — write them like prompts, including when to use and what parameters mean.
2. `is_error: false` for a successful call with zero results; `true` only when execution itself failed.
3. MCP separates host (app), client (protocol connector), and server (tool/resource/prompt provider) as real process boundaries.
4. Scope each agent to ~4-5 relevant tools, never the full catalog — this improves both selection accuracy and security.
5. Tool results are data to reason about, not instructions to obey — treat them as untrusted input, same as any other context.

KEY TERMS TO RECALL UNAIDED

tool_choice: auto any tool none is_error MCP host MCP client MCP server resources
prompts stdio HTTP transport least privilege

Claude Code Configuration & Workflows

IF YOU REMEMBER ONLY FIVE THINGS

1. Path prefix tells you everything: `~/.claude/` = personal, never git-tracked; `.claude/` in a repo = shared via git.
2. `.mcp.json` = project-shared MCP servers; `~/.claude.json` = personal, cross-project MCP servers.
3. `CLAUDE.md` gives durable, auto-loaded project memory — commands, conventions, structure, gotchas — layerable by directory.
4. `allowedTools` /deny-lists in `settings.json` are deterministic, host-enforced permission boundaries.
5. Headless mode (`-p` flag) plus a scoped allow-list is what makes Claude Code safe and usable inside CI/CD.

KEY TERMS TO RECALL UNAIDED

CLAUDE.md settings.json .mcp.json ~/.claude.json allowedTools context: fork slash commands
headless mode -p flag Claude Agent SDK

Prompt Engineering & Structured Output

IF YOU REMEMBER ONLY FIVE THINGS

1. Explicit, enumerable criteria beat vague qualifiers like "be conservative" — consistency requires a checkable rule.
2. Few-shot examples earn their keep on borderline/ambiguous cases; they add little to already-correct obvious cases.
3. Tool-forced structured output guarantees shape and syntax — it never guarantees the semantic correctness of a value.
4. A retry/validation loop fixes malformed output; it cannot fix an answer that's wrong because the fact was never in the source.
5. Batch API: ~50% cheaper, asynchronous, no multi-turn tool calling — bulk/non-urgent only, never for live interactive agents.

KEY TERMS TO RECALL UNAIDED

system prompt categorical criteria few-shot chain-of-thought extended thinking
tool-forced output syntax vs semantics validation loop prompt caching Batch API

Context Management & Reliability

IF YOU REMEMBER ONLY FIVE THINGS

1. The context window is a finite, shared resource across prompt, history, tools, and retrieved content — manage it actively.
2. Compaction saves tokens but must never touch a separate, verbatim case-facts block of exact IDs/amounts/dates.
3. Lost-in-the-middle: place the most important finding at the start of long context, not buried in the middle.
4. Conflicting sources get surfaced and flagged as contested — never silently picked between or blended.
5. Three escalation triggers: explicit human request, policy gap, or inability to make progress — always escalate, never guess.

KEY TERMS TO RECALL UNAIDED

context window compaction case-facts block lost-in-the-middle grounding escalation trigger
RAG / retrieval chunking

Production-Ready Reference Templates

Copyable starting points that apply every principle in Chapters 1–9 correctly

These templates are teaching artifacts, not sample exam questions — each one is a correct, defensible implementation of the patterns from Chapter 10, annotated so you can see exactly which principle justifies each line. Studying *why* a template is shaped the way it is reinforces recall better than re-reading the prose explanation a second time.

13.1 Template: A Least-Privilege Support Agent System Prompt

You are a Tier-1 customer support agent for Acme Retail.

SCOPE: You may look up orders, check shipping status, and issue refunds under \$50 for orders placed within 30 days with no more than one prior refund in the past 12 months. You may not discuss other customers' accounts, modify prices, or promise delivery dates the shipping system has not confirmed.

ESCALATE (do not attempt to resolve yourself) when:

- The customer explicitly asks for a human.
- A refund request falls outside the criteria above.
- You cannot retrieve the information needed to proceed after one reasonable retry.

When sources disagree (e.g., stated delivery date vs. system status), state both and flag the discrepancy — do not guess which one is correct.

Annotations: the SCOPE paragraph is an explicit categorical criterion (5.1) — no "use good judgment" language remains. The ESCALATE list implements the three canonical triggers exactly (6.4). The final paragraph implements the flag-don't-resolve pattern (Ch.10 #13) directly in the prompt layer, complementing whatever hooks enforce the refund ceiling deterministically (2.5).

13.2 Template: A Scoped CI/CD settings.json

```
{
  "allowedTools": [
    "Bash(npm ci:*)", "Bash(npm test:*)",
    "Bash(npm run lint:*)", "Read"
  ],
  "deniedTools": ["Bash(git push:*)", "Bash(rm:*)", "Write"],
  "maxIterations": 15,
  "timeoutSeconds": 300,
  "hooks": {
    "PostToolUse": [{ "matcher": "*", "command": "scripts/ci_audit_log.sh" }]
  }
}
```

Annotations: this is the scoped headless automation pattern (Ch.10 #15) in full: an explicit allow-list scoped to exactly what a read-only CI check needs, a deny-list making destructive commands unreachable, a hard iteration cap and timeout (2.7), and a `PostToolUse` audit hook satisfying any compliance/traceability requirement (2.5).

13.3 Template: A Subagent Result Contract Schema

```
{
  "type": "object",
  "properties": {
    "subagent_id": {"type": "string"},
    "status": {"type": "string", "enum": ["success", "partial_success", "failed"]},
    "summary": {"type": "string"},
    "unresolved": {"type": "array", "items": {"type": "string"}},
    "sources": {"type": "array", "items": {"type": "string"}}
  },
  "required": ["subagent_id", "status", "summary"]
}
```

Annotations: forcing this shape via `tool_choice` (3.2, 5.4) on every subagent's final report guarantees the orchestrator always receives a machine-checkable status rather than inferring completeness from prose — directly implementing the structured result contract pattern (Ch.10 #4) and enabling the grounding techniques from 6.5 through the `sources` field.

13.4 Template: A Case-Facts-Preserving Compaction

Instruction

When compacting conversation history, produce exactly two sections:

CASE_FACTS (JSON, verbatim, never summarized): every ID, dollar amount, date, and explicit decision already made in this session.

NARRATIVE_SUMMARY (prose, may be condensed freely): the general flow of the conversation, excluding anything already captured in CASE_FACTS above.

Never let a value that belongs in CASE_FACTS appear only inside the narrative summary.

Annotations: this is the case-facts preservation pattern (Ch.10 #11, taught fully in 6.2 and 6.6) expressed as a reusable compaction instruction – the explicit final sentence closes the most common failure mode, where a specific value quietly migrates into prose and becomes vulnerable to later summarization drift.